



دانشگاه شاهد

Shahed University

Turing Machine

Student: Matin Moradi

Master : Vahid Haghighatdoost

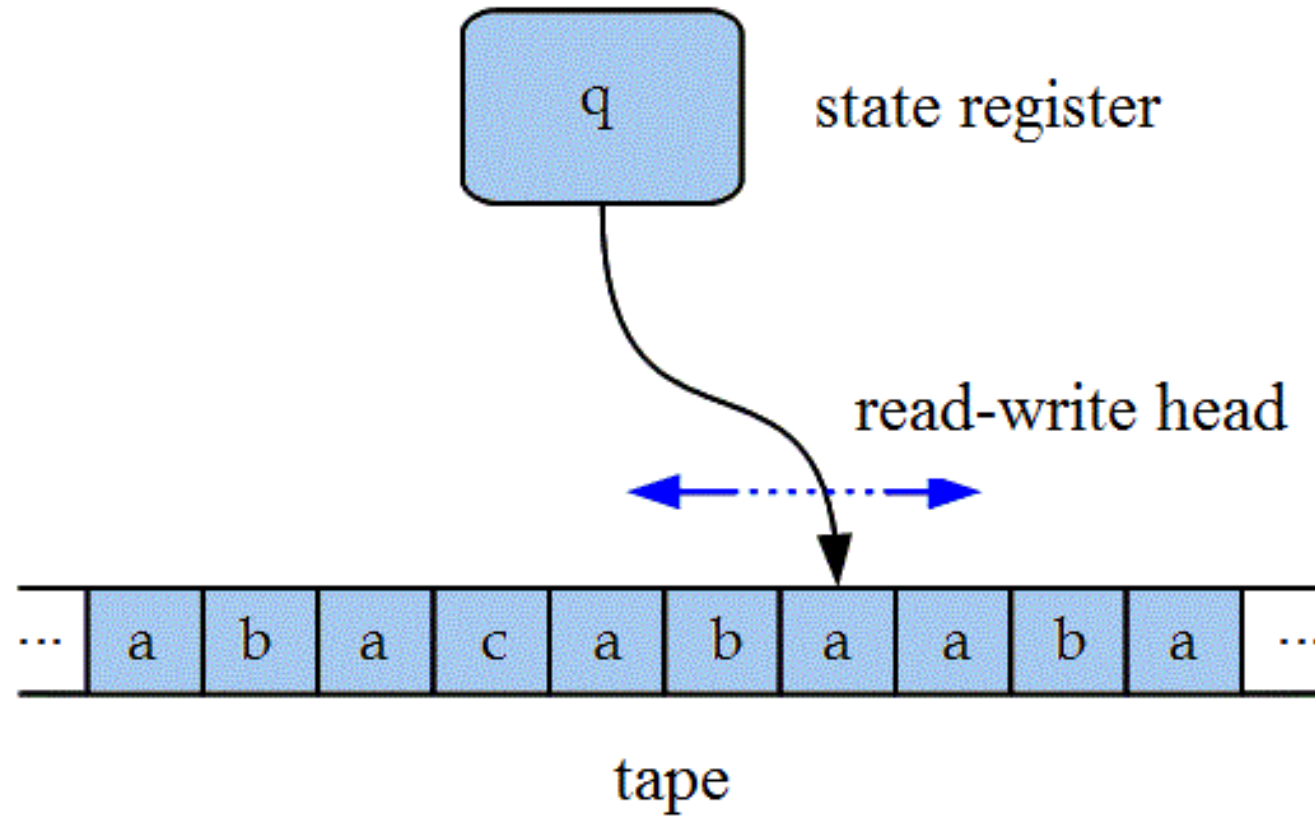
Headline

- Turing Machine
- Turing Thesis
- 2D Turing Machine
- Neural Turing Machine

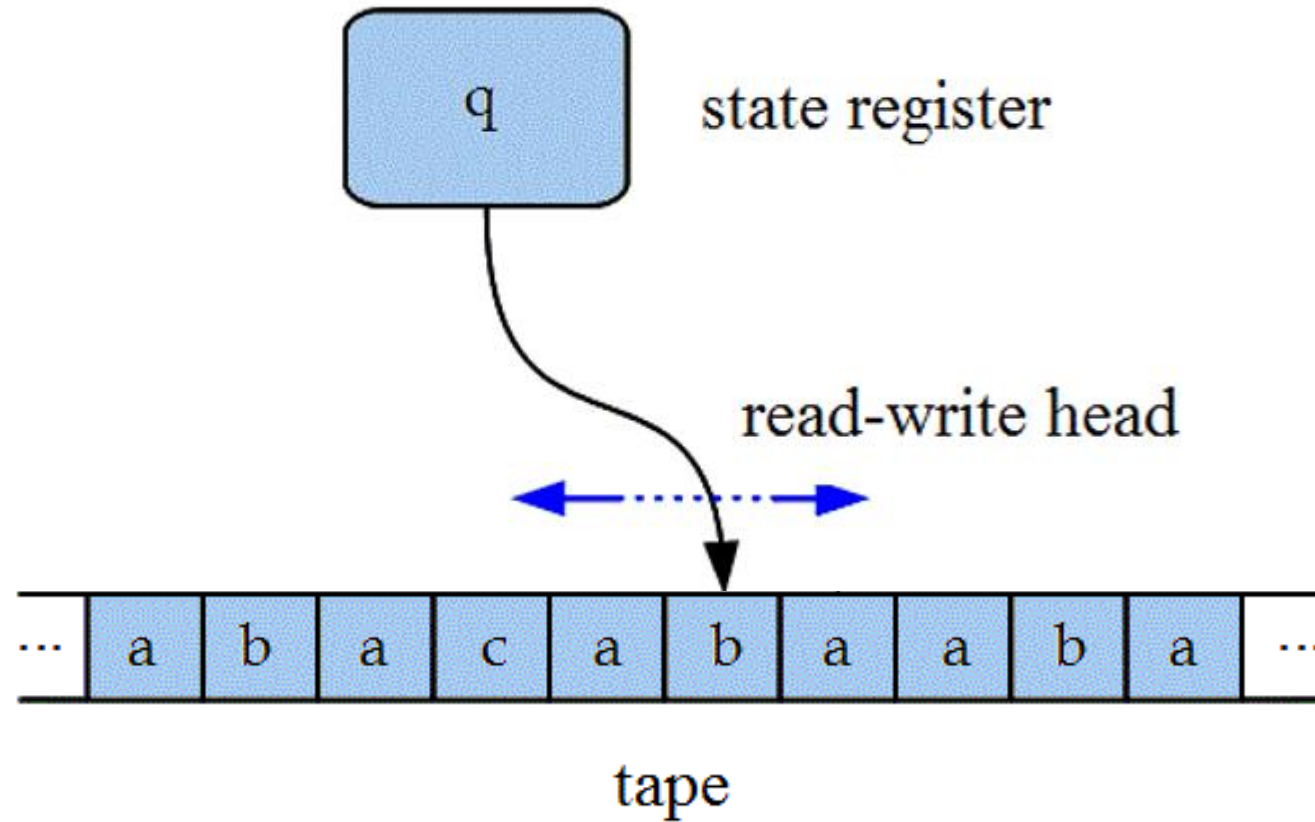


Move

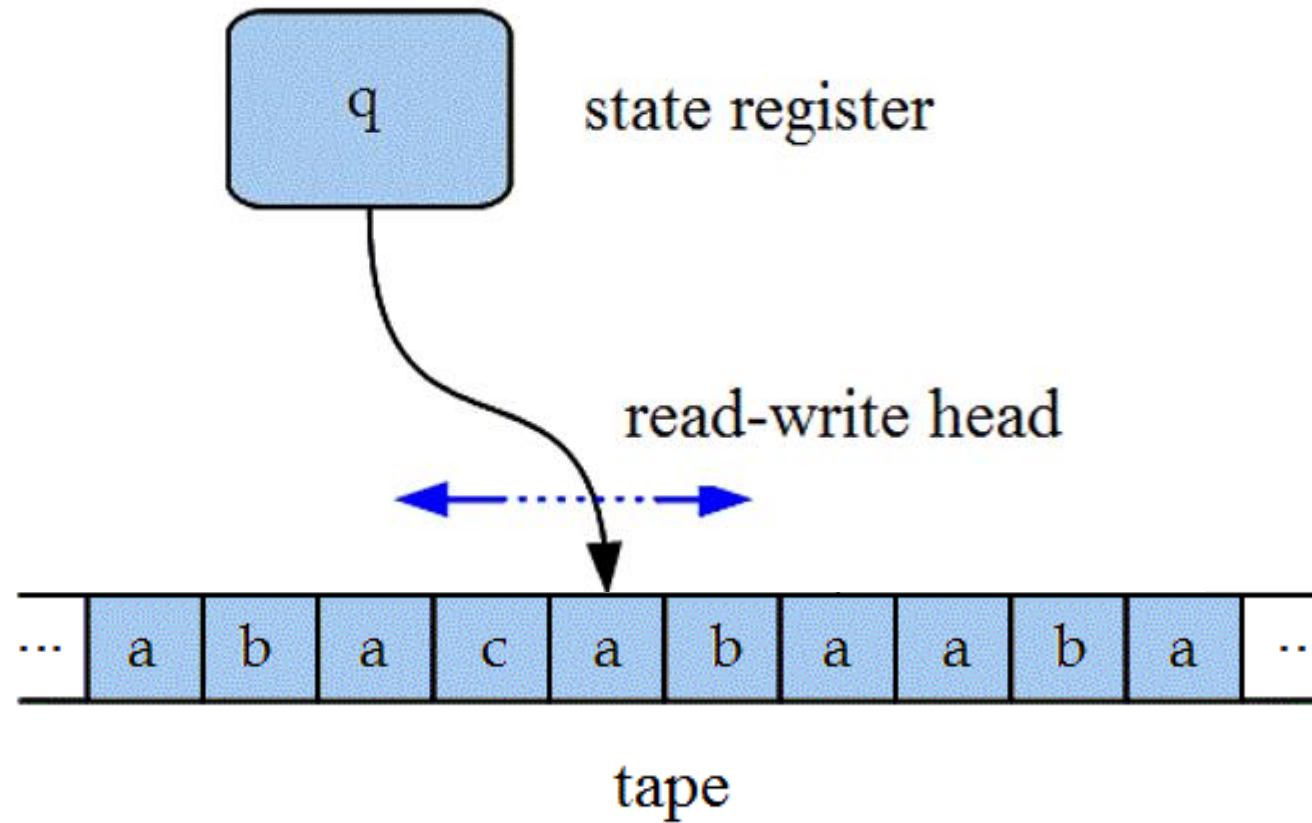
Definition



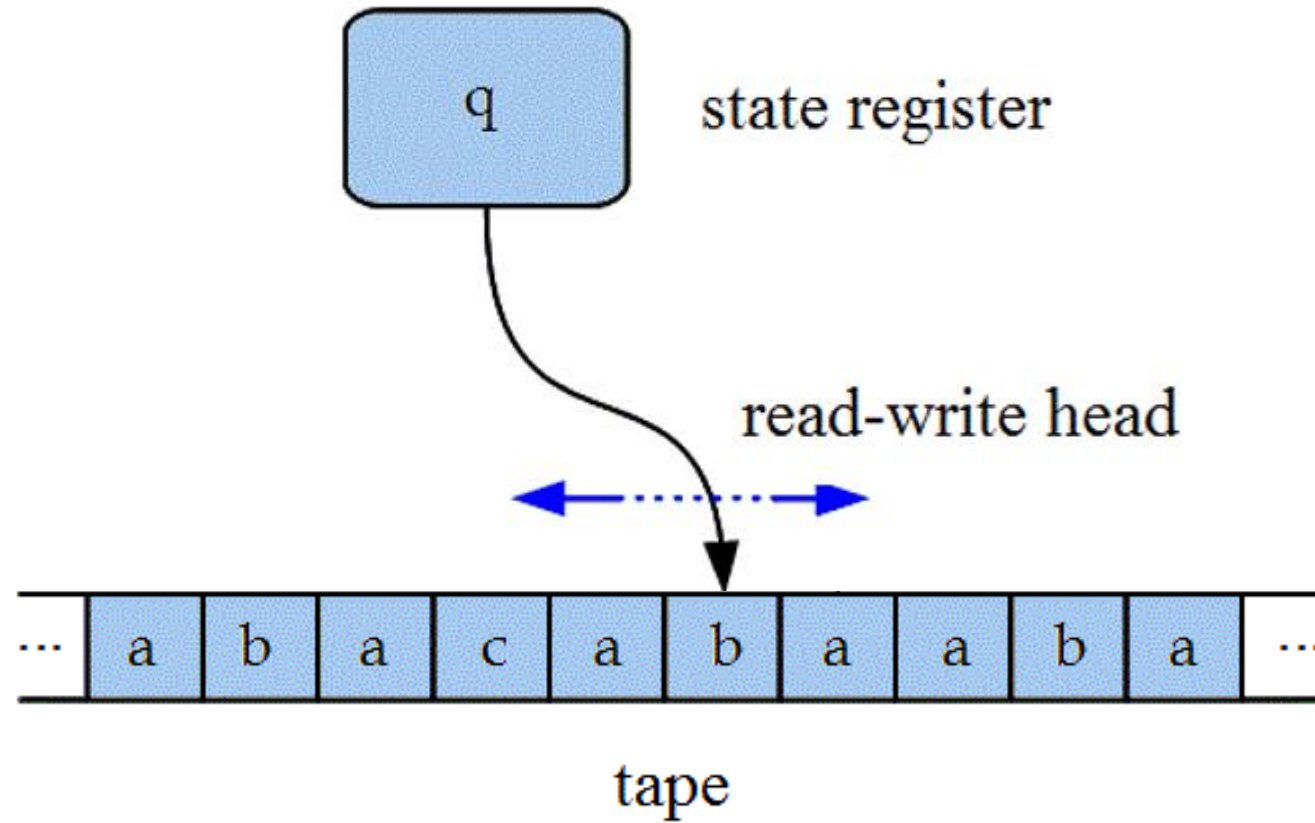
Definition



Definition

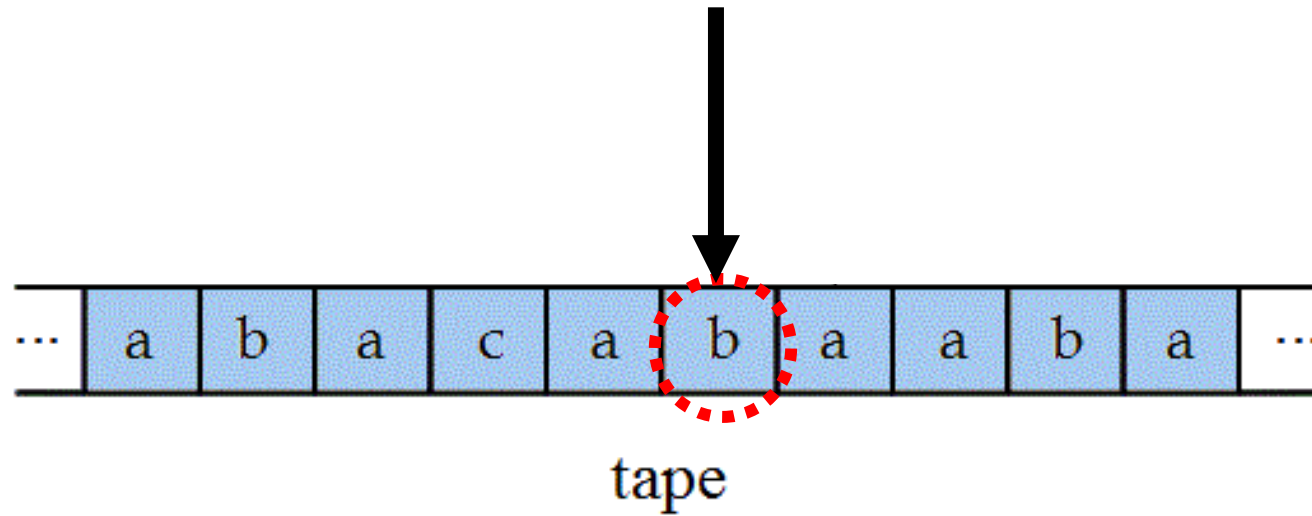


Definition

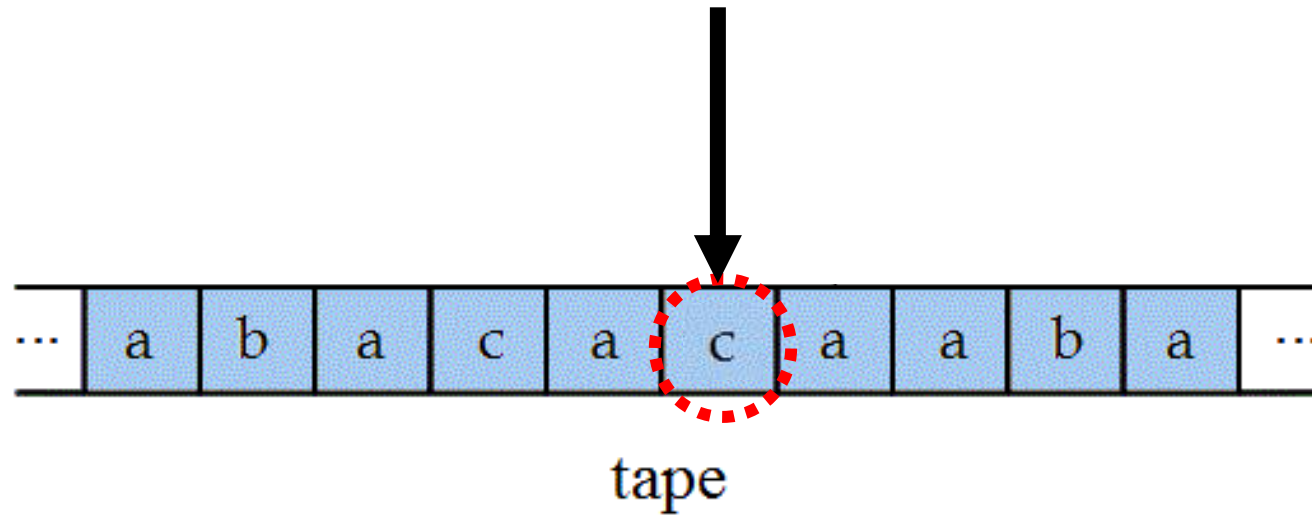


Change

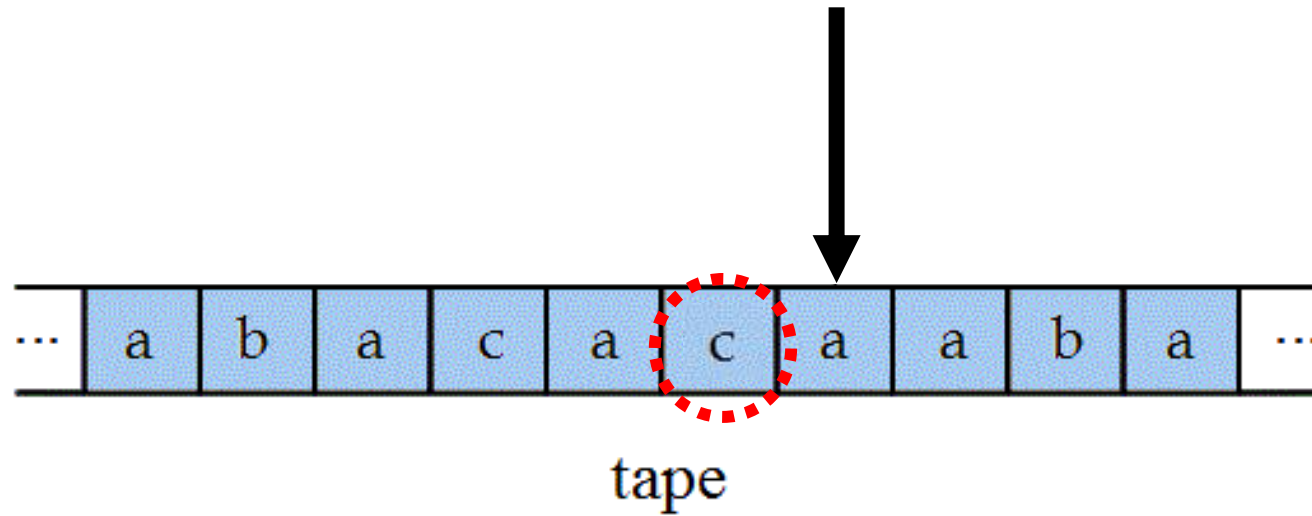
Definition



Definition

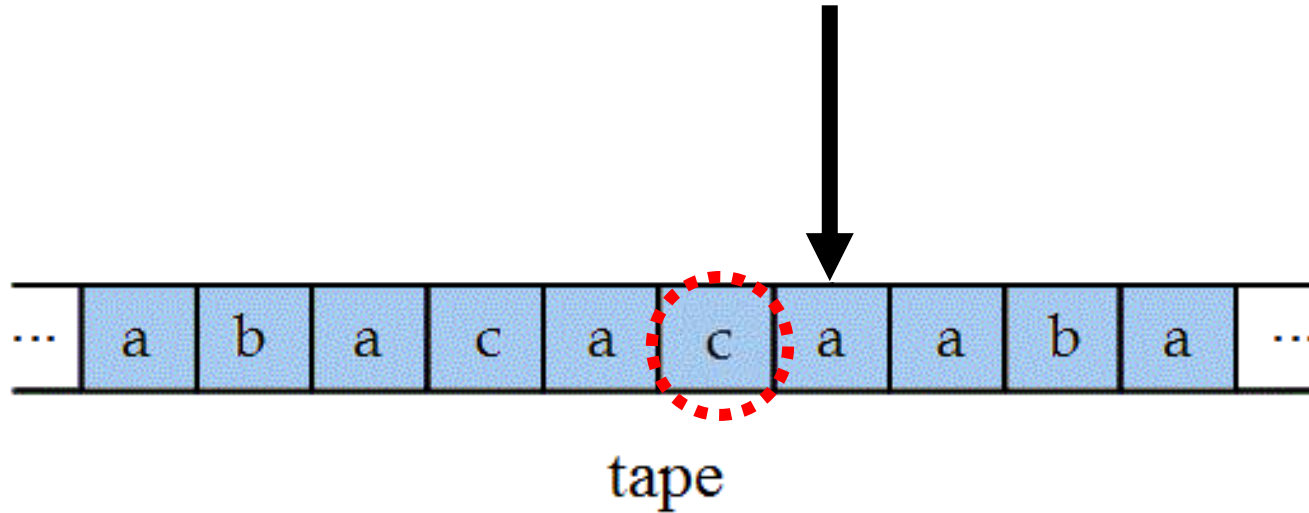


Definition



Definition

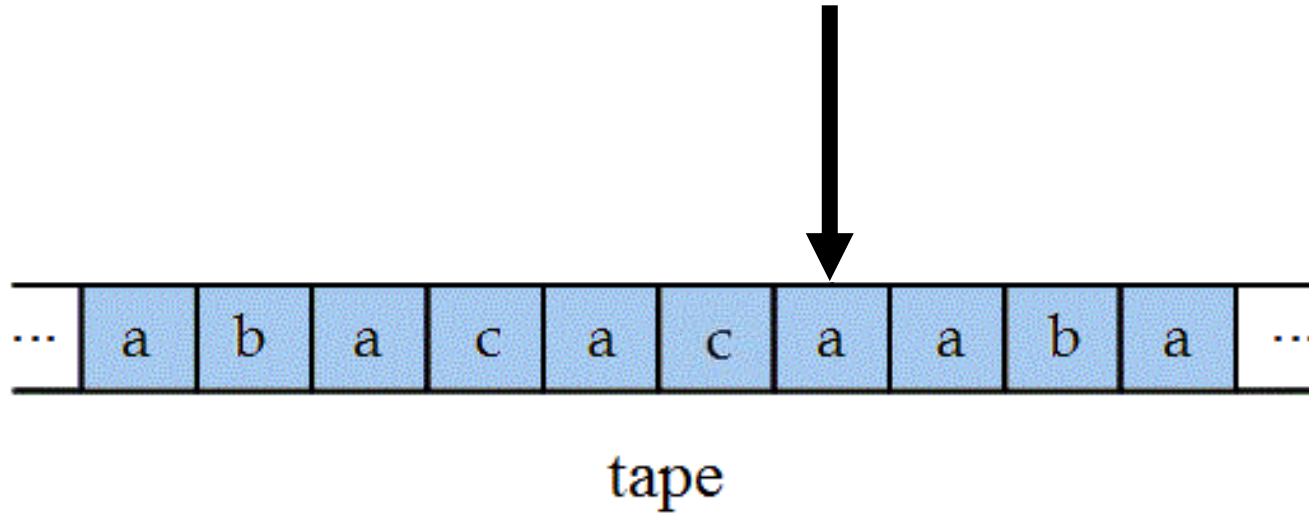
$$\delta(q_0, b) = (q_1, c, R)$$



Example

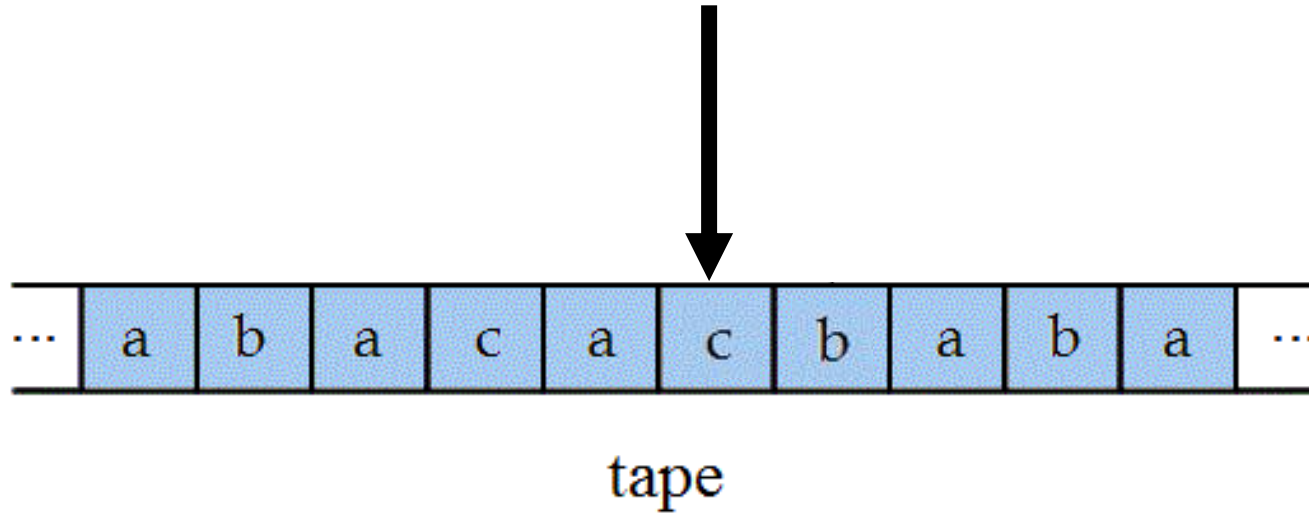
Definition

$$\delta(q_1, a) = (q_1, b, L)$$



Definition

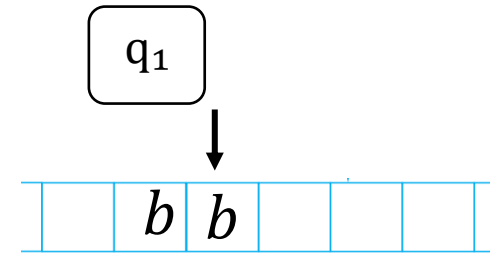
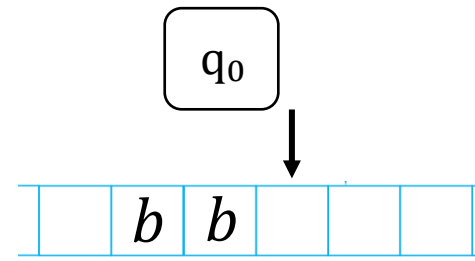
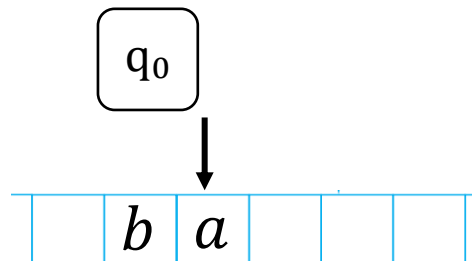
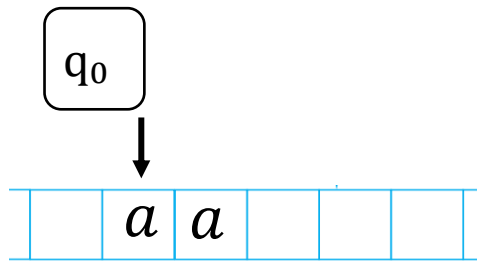
$$\delta(q_1, a) = (q_1, b, L)$$



Example 9.2

Example 9.2

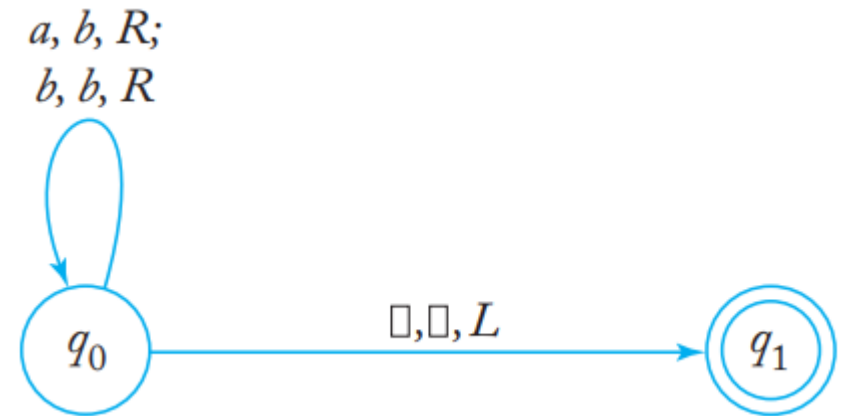
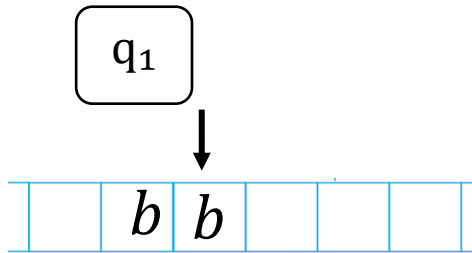
$$\begin{aligned}\delta(q_0, a) &= (q_0, b, R), \\ \delta(q_0, b) &= (q_0, b, R), \\ \delta(q_0, \square) &= (q_1, \square, L)\end{aligned}$$



Example 9.2

$$\begin{aligned}\delta(q_0, a) &= (q_0, b, R), \\ \delta(q_0, b) &= (q_0, b, R), \\ \delta(q_0, \square) &= (q_1, \square, L)\end{aligned}$$

- Halt State
- Program
- $q_0aa \vdash^* \infty$
- Transition Graph



Definition

Definition (cont.)

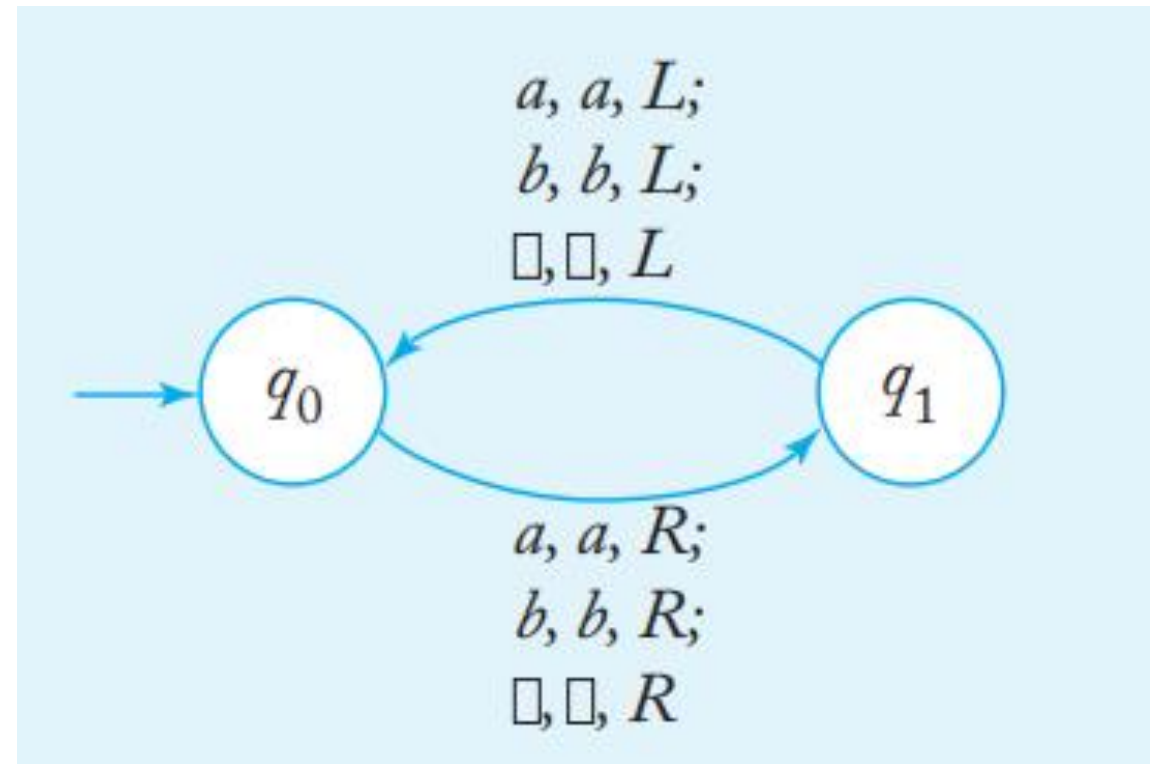
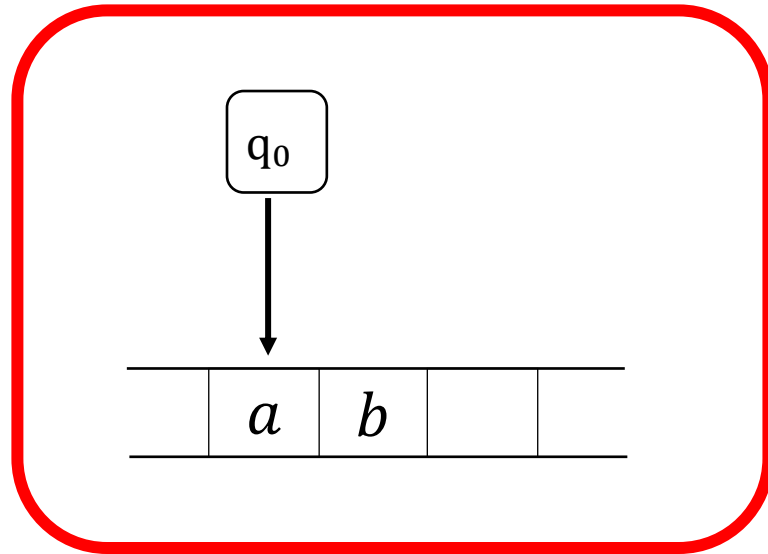
- A Turing machine M is defined by

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \sqcup, F)$$

$$\Sigma \subseteq \Gamma - \{\sqcup\}$$
$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

- Q is the **set of internal states**
- Σ is the **input** alphabet
- Γ is a finite set of symbols called the **tape alphabet**
- δ is the **transition function**
- $\sqcup \in \Gamma$ is a special symbol called the **blank**
- $q_0 \in Q$ is the **initial state**
- $F \subseteq Q$ is the **set of final states**

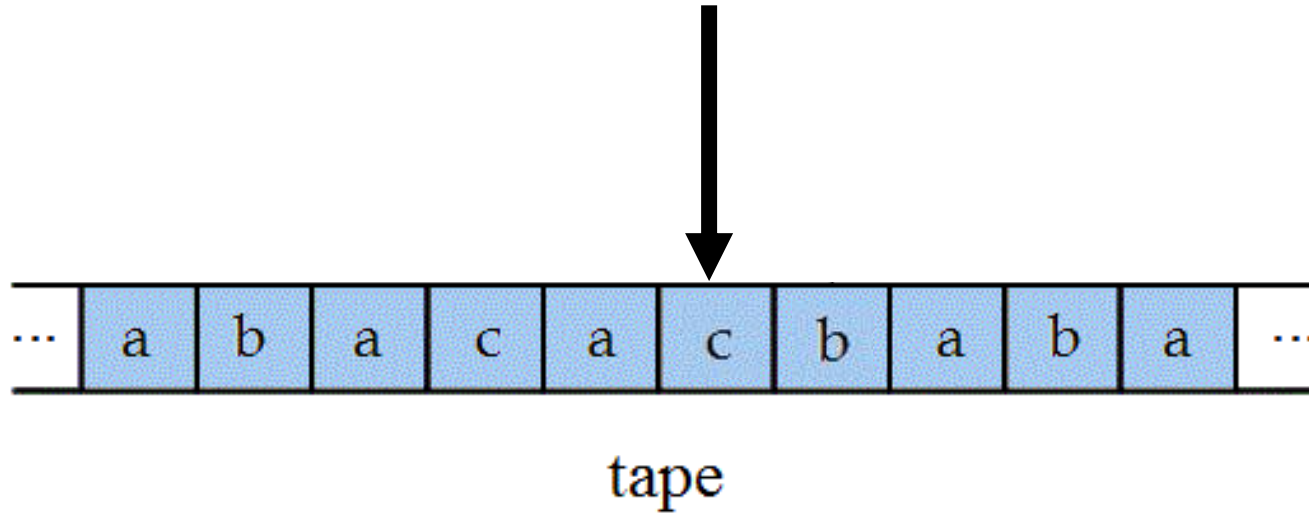
Example 9.3



Push Down Automata Vs Turing Machine

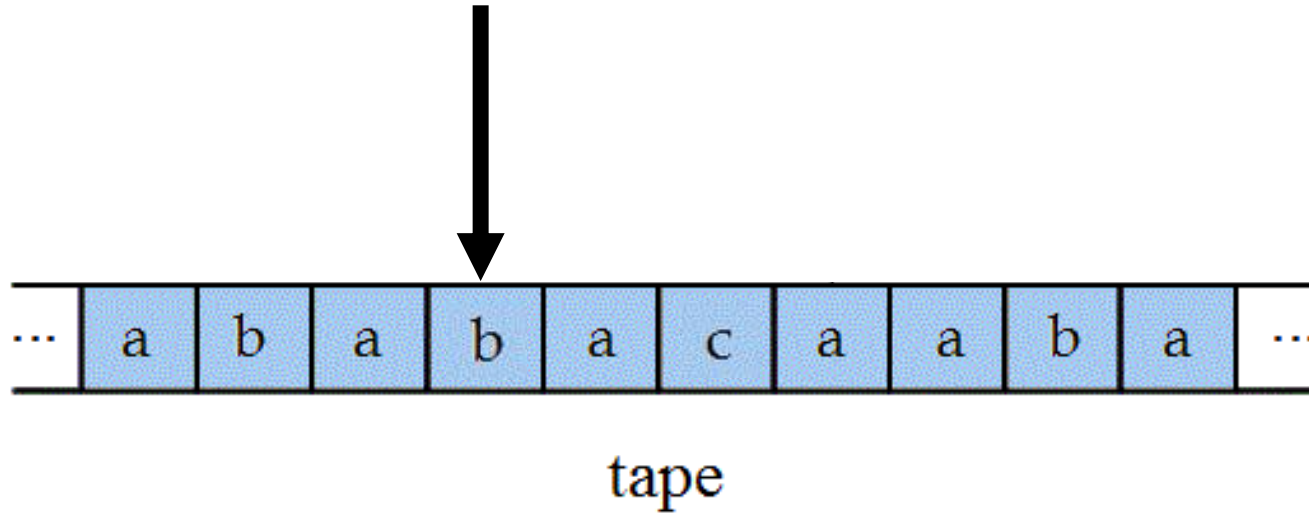
instantaneous description

- *abaca* *q* *cbaba*



instantaneous description

- *aba* ***q*** *bacaaba*



instantaneous description

- $a \sqcap a \textcolor{red}{q} b a \sqcap a a b a$

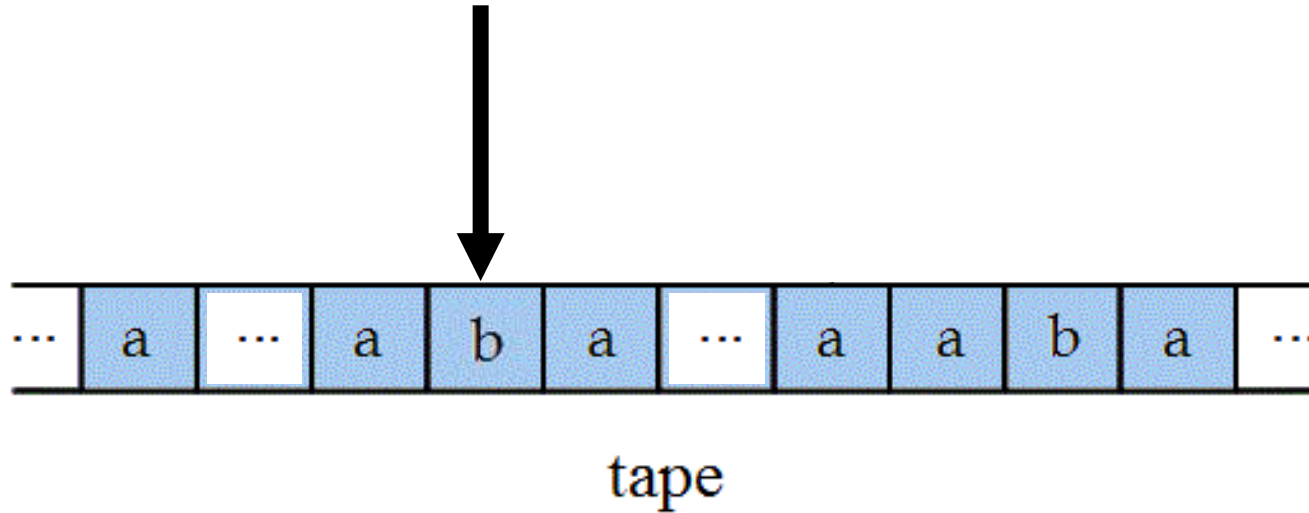
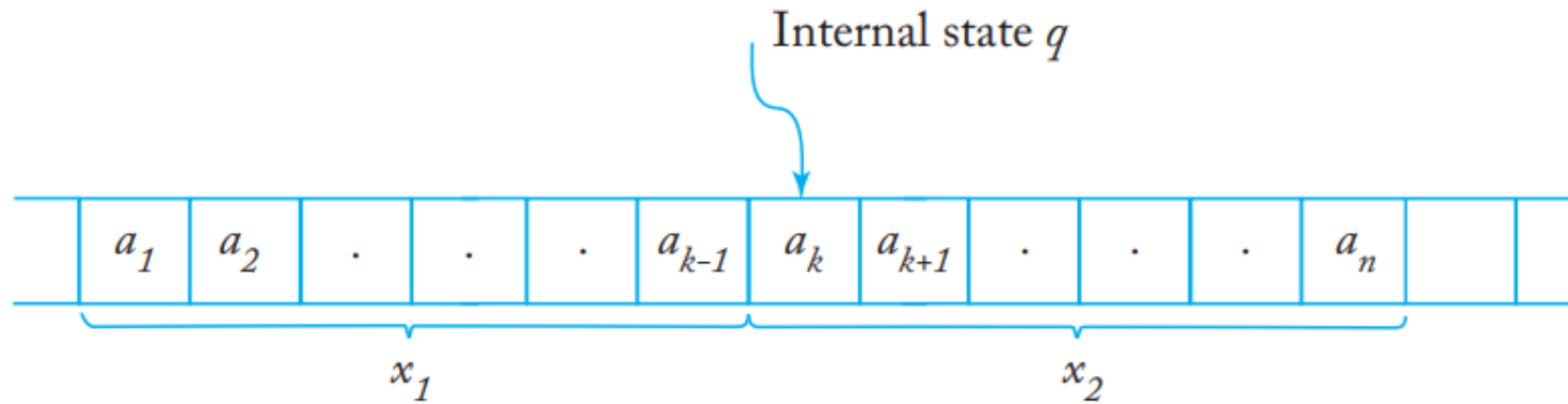


Figure 9.6

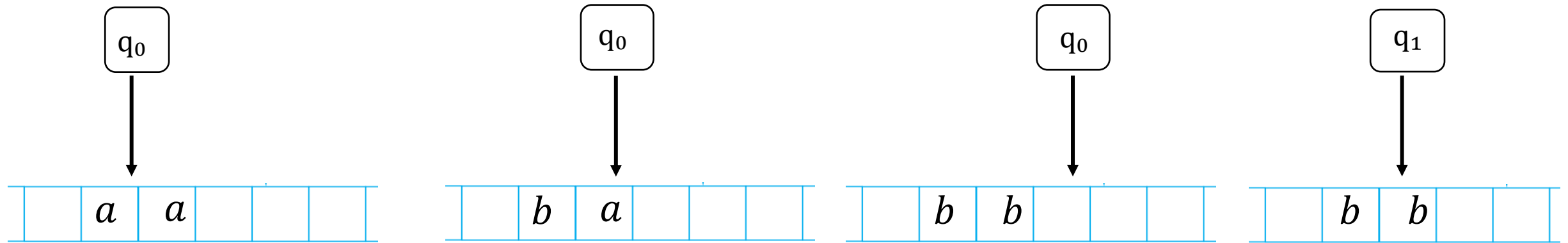
Figure 9.6

- $a_1 a_2 \dots a_{k-1} \textcolor{red}{q} a_k \dots a_n$



Example 9.4

Example 9.4



$q_0aa \vdash bq_0a \vdash bbq_0\Box \vdash bq_1b$

$q_0aa \vdash^* bq_1b$

$\vdash_M$

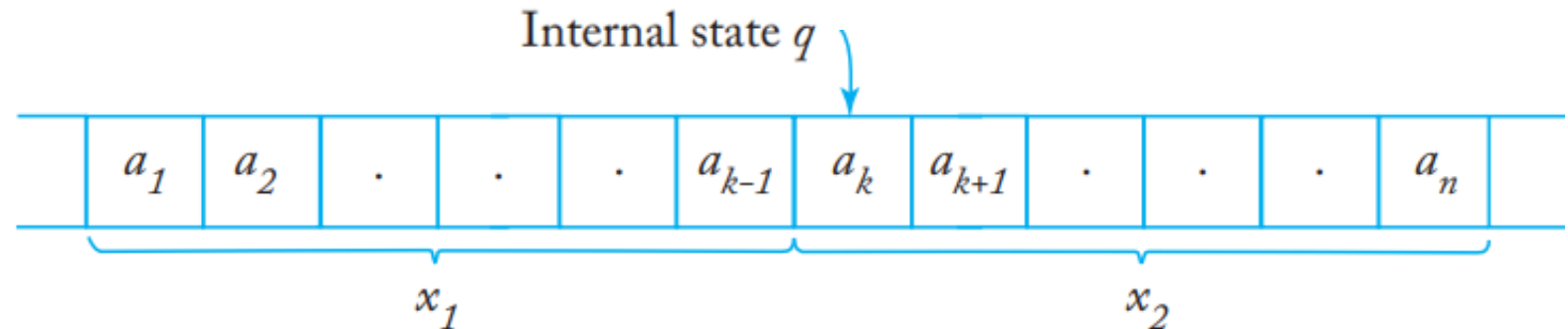
Definition 9.2

Computation

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F) \quad | \quad a_i \in \Gamma \quad | \quad q_1 \in Q$$

$$1. \delta(q_1, a_k) = (q_2, b, R)$$

$$2. \delta(q_1, a_k) = (q_2, b, L)$$



Turing Machine as Acceptor

Language Accepters

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$$

$$L(M) = \{w \in \Sigma^+ : q_0 w \vdash^* x_1 q_f x_2 \text{ for some } q_f \in F, x_1, x_2 \in \Gamma^* \}$$

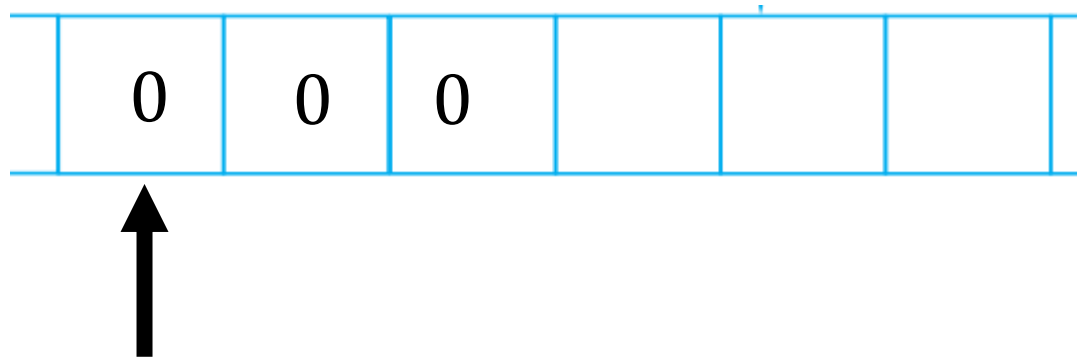
1. $w \in L(M)$
2. $w \notin L(M)$
 - Halt in nonfinal State
 - Enter in infinite Loop

Example 9.6

Example 9.6

- For $\Sigma = \{0, 1\}$, design a Turing machine that accepts the language denoted by the regular expression 00^* .

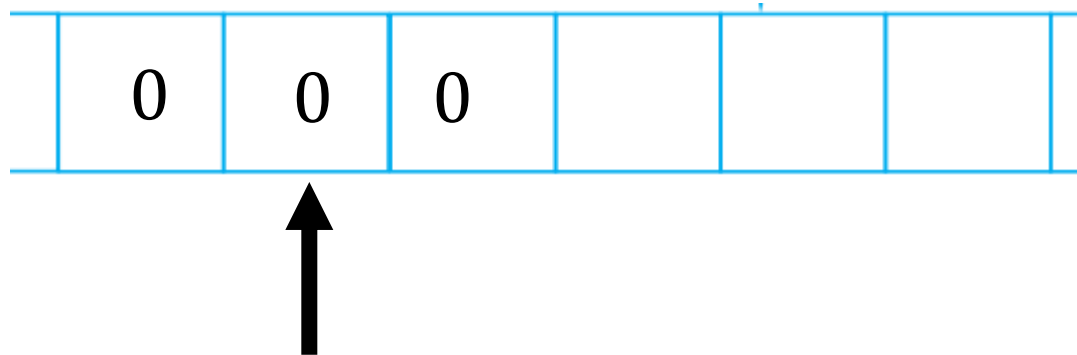
$$\begin{aligned}\delta(q_0, 0) &= (q_0, 0, R), \\ \delta(q_0, \square) &= (q_1, \square, R)\end{aligned}$$



Example 9.6

- For $\Sigma = \{0, 1\}$, design a Turing machine that accepts the language denoted by the regular expression 00^* .

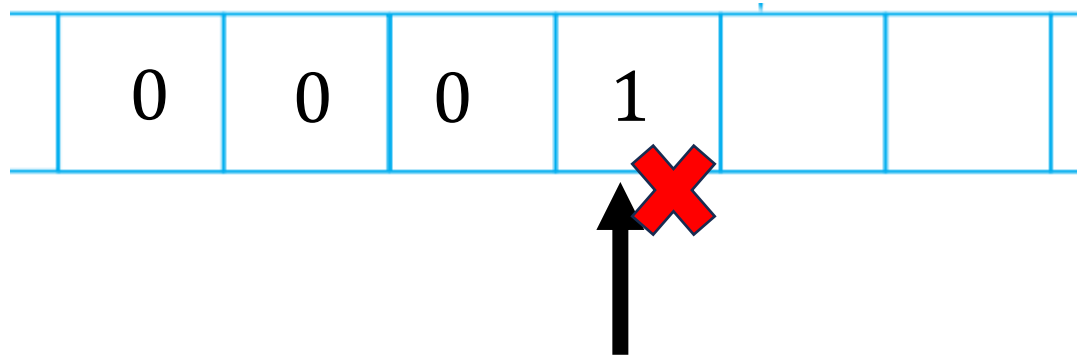
$$\begin{aligned}\delta(q_0, 0) &= (q_0, 0, R), \\ \delta(q_0, \square) &= (q_1, \square, R)\end{aligned}$$



Example 9.6

- For $\Sigma = \{0, 1\}$, design a Turing machine that accepts the language denoted by the regular expression 00^* .

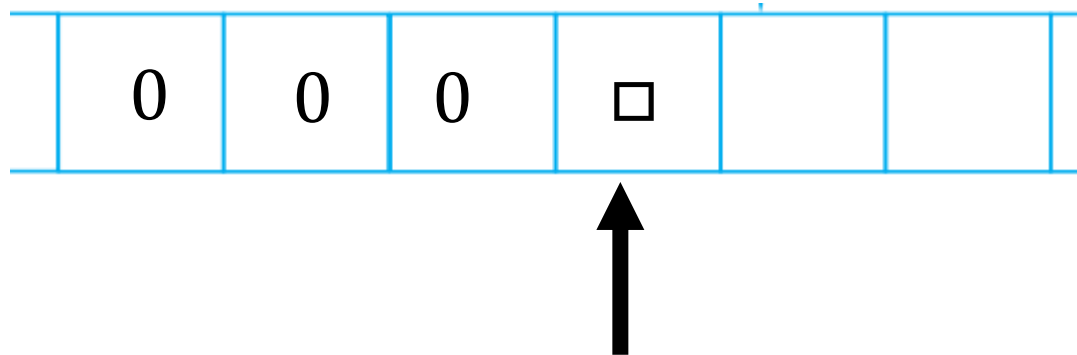
$$\begin{aligned}\delta(q_0, 0) &= (q_0, 0, R), \\ \delta(q_0, \square) &= (q_1, \square, R)\end{aligned}$$



Example 9.6

- For $\Sigma = \{0, 1\}$, design a Turing machine that accepts the language denoted by the regular expression 00^* .

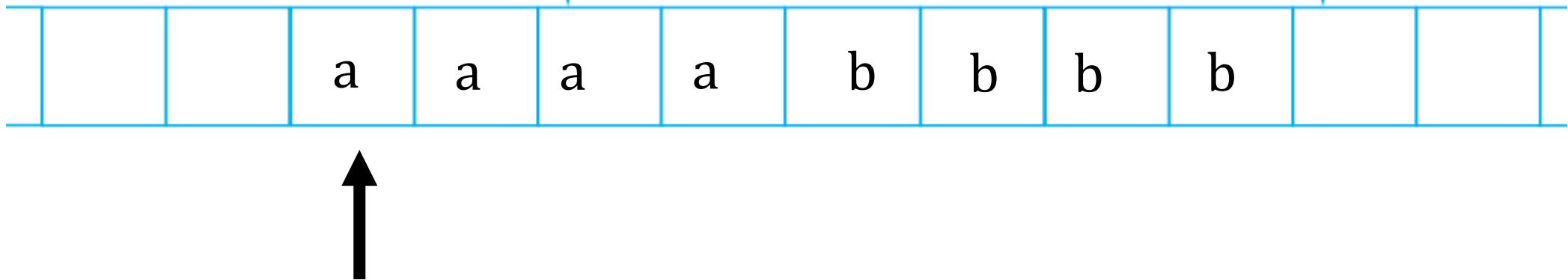
$$\begin{aligned}\delta(q_0, 0) &= (q_0, 0, R), \\ \delta(q_0, \square) &= (q_1, \square, R)\end{aligned}$$



Example 9.7

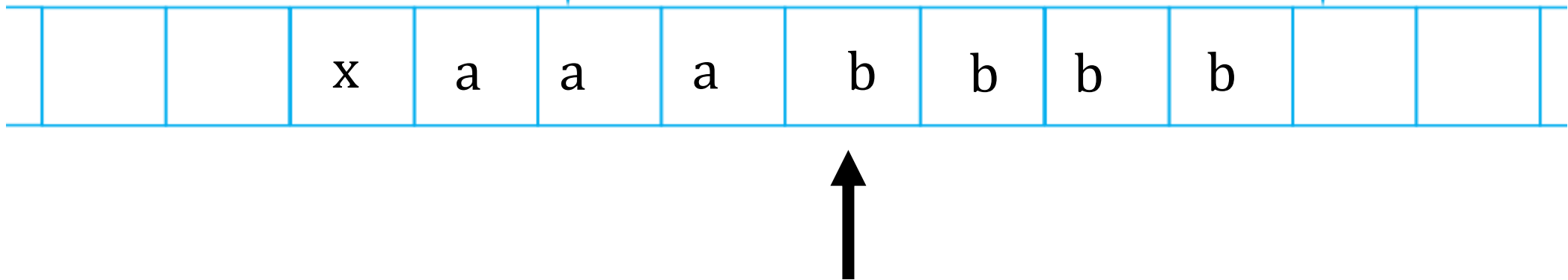
Example 9.7

- For $\Sigma = \{a, b\}$, design a Turing machine that accepts $L = \{a^n b^n : n \geq 1\}$.



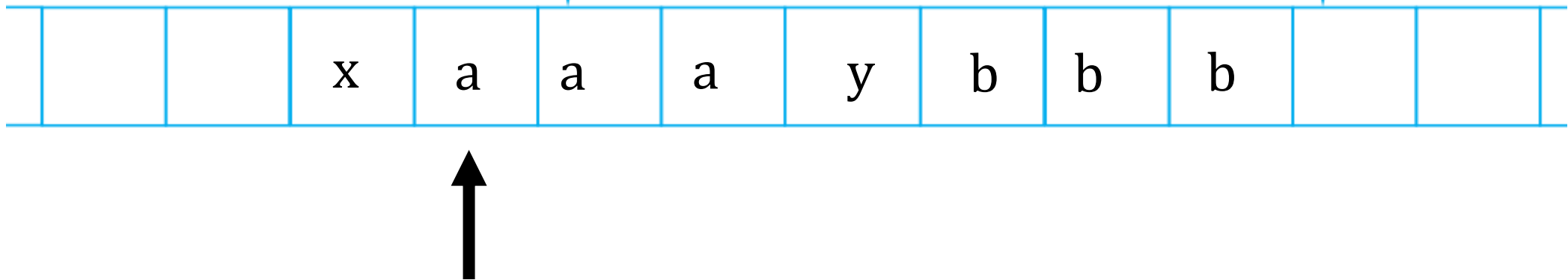
Example 9.7

- For $\Sigma = \{a, b\}$, design a Turing machine that accepts $L = \{a^n b^n : n \geq 1\}$.



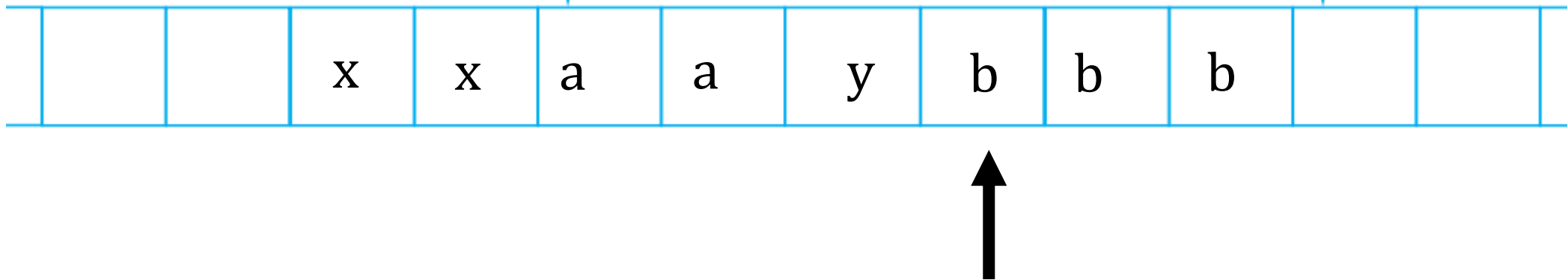
Example 9.7

- For $\Sigma = \{a, b\}$, design a Turing machine that accepts $L = \{a^n b^n : n \geq 1\}$.



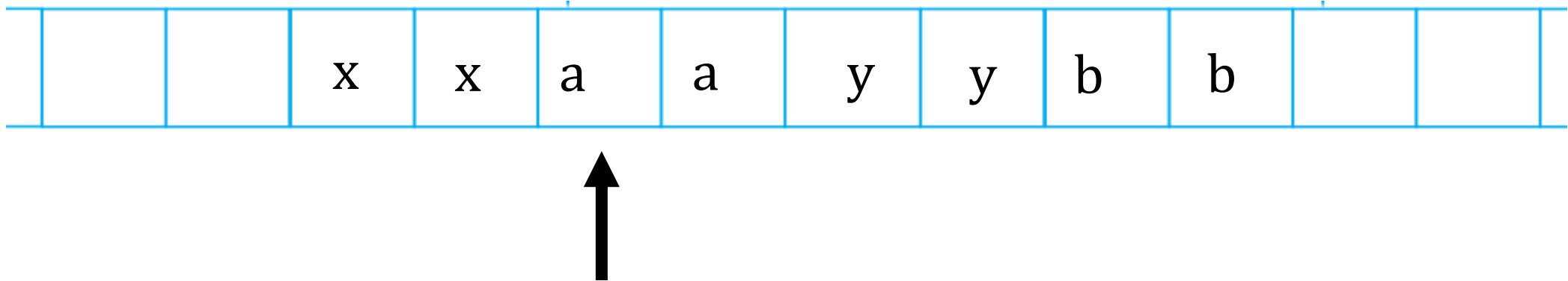
Example 9.7

- For $\Sigma = \{a, b\}$, design a Turing machine that accepts $L = \{a^n b^n : n \geq 1\}$.



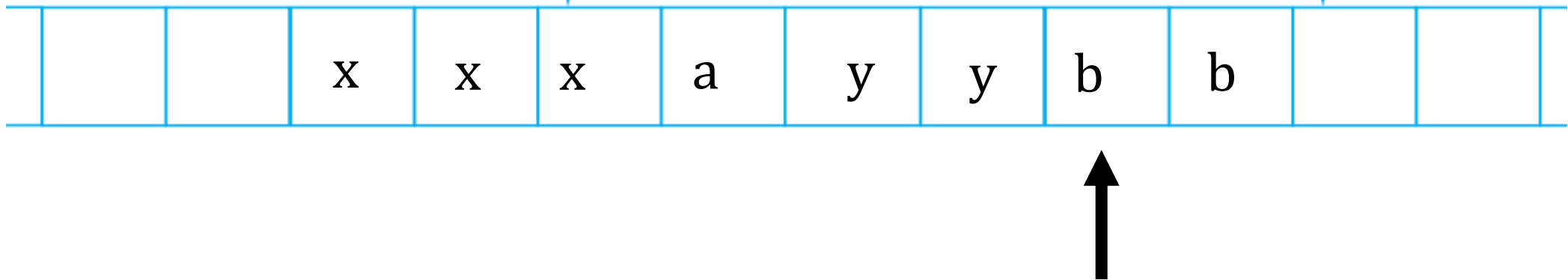
Example 9.7

- For $\Sigma = \{a, b\}$, design a Turing machine that accepts $L = \{a^n b^n : n \geq 1\}$.



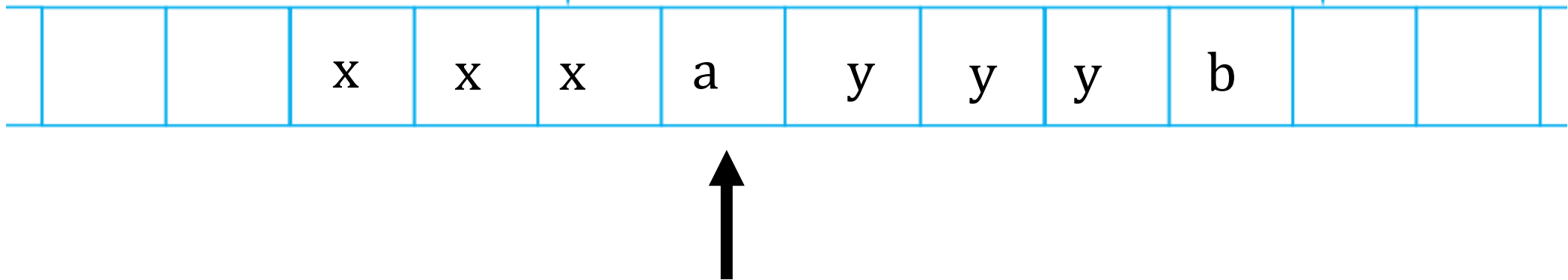
Example 9.7

- For $\Sigma = \{a, b\}$, design a Turing machine that accepts $L = \{a^n b^n : n \geq 1\}$.



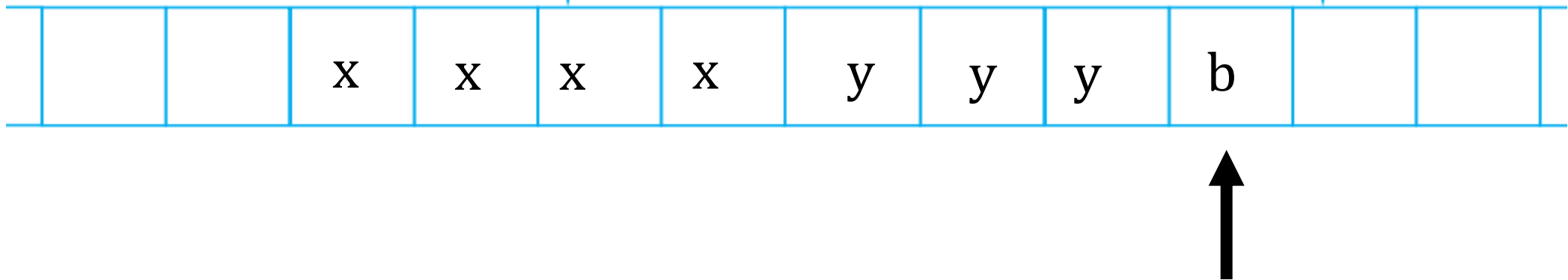
Example 9.7

- For $\Sigma = \{a, b\}$, design a Turing machine that accepts $L = \{a^n b^n : n \geq 1\}$.



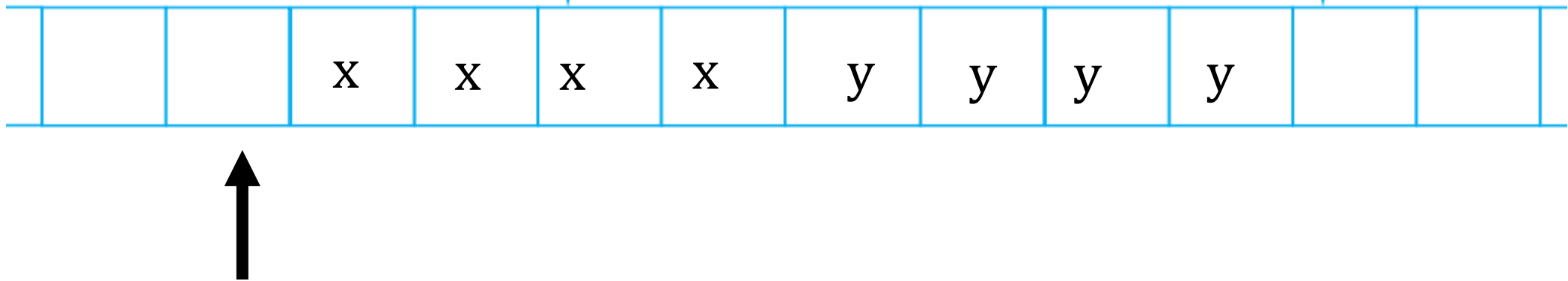
Example 9.7

- For $\Sigma = \{a, b\}$, design a Turing machine that accepts $L = \{a^n b^n : n \geq 1\}$.



Example 9.7

- For $\Sigma = \{a, b\}$, design a Turing machine that accepts $L = \{a^n b^n : n \geq 1\}$.



Turing Machine

Example 9.7

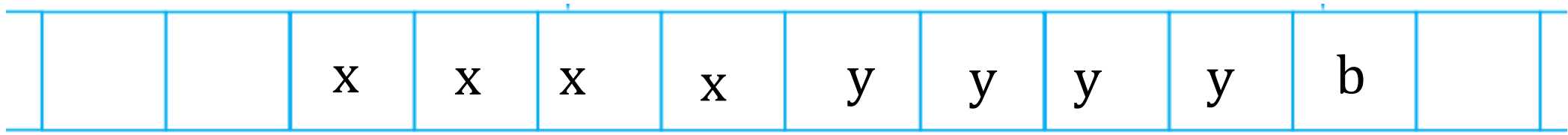
- For $\Sigma = \{a, b\}$, design a Turing machine that accepts $L = \{a^n b^n : n \geq 1\}$.



Turing Machine

Example 9.7

- For $\Sigma = \{a, b\}$, design a Turing machine that accepts $L = \{a^n b^n : n \geq 1\}$.



Turing Machine

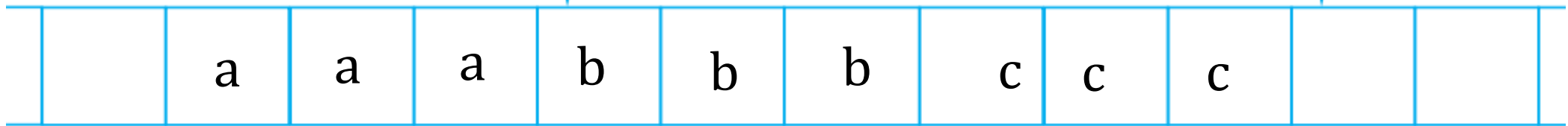
Example 9.7

$$\begin{aligned}\delta(q_0, a) &= (q_1, x, R), \\ \delta(q_1, a) &= (q_1, a, R), \\ \delta(q_1, y) &= (q_1, y, R), \\ \delta(q_1, b) &= (q_2, y, L)\end{aligned}$$
$$\begin{aligned}\delta(q_2, y) &= (q_2, y, L), \\ \delta(q_2, a) &= (q_2, a, L), \\ \delta(q_2, x) &= (q_0, x, R),\end{aligned}$$
$$\begin{aligned}\delta(q_0, y) &= (q_3, y, L), \\ \delta(q_3, y) &= (q_3, y, R), \\ \delta(q_3, \square) &= (q_4, \square, R),\end{aligned}$$

Example 9.8

Example 9.8

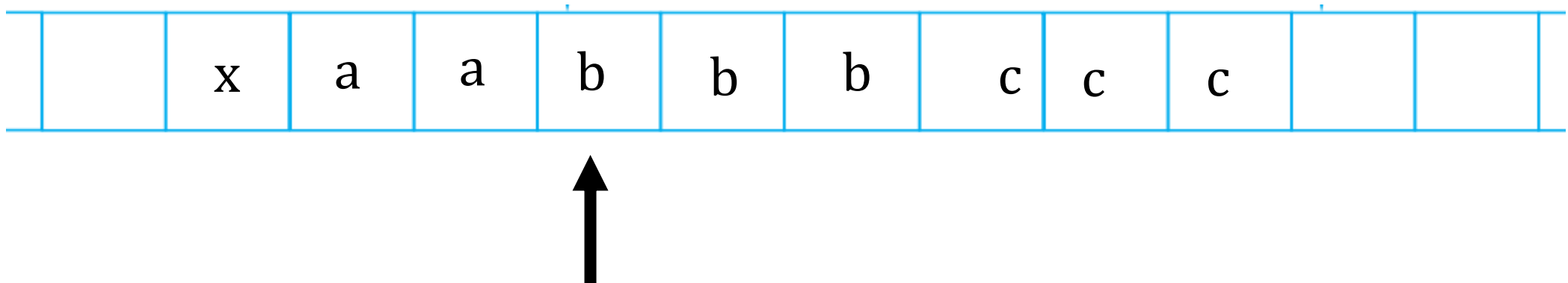
- For $\Sigma = \{a, b, c\}$, design a Turing machine that accepts $L = \{a^n b^n c^n : n \geq 1\}$.



Turing Machine

Example 9.8

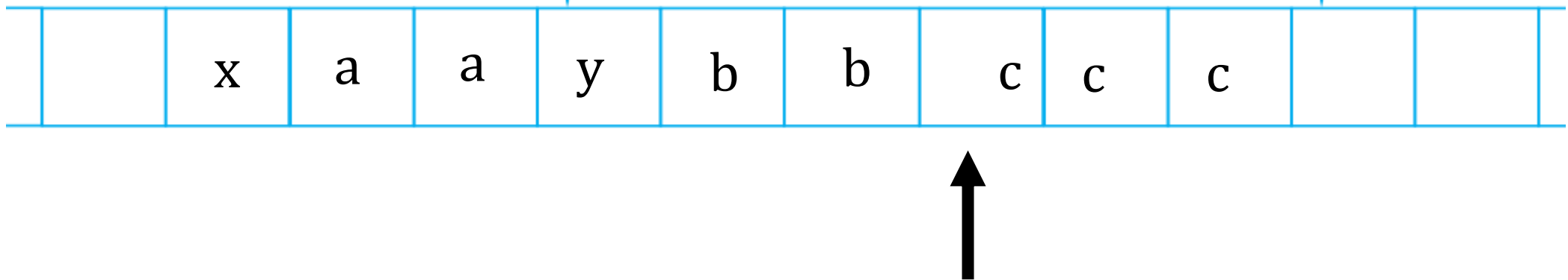
- For $\Sigma = \{a, b, c\}$, design a Turing machine that accepts $L = \{a^n b^n c^n : n \geq 1\}$.



Turing Machine

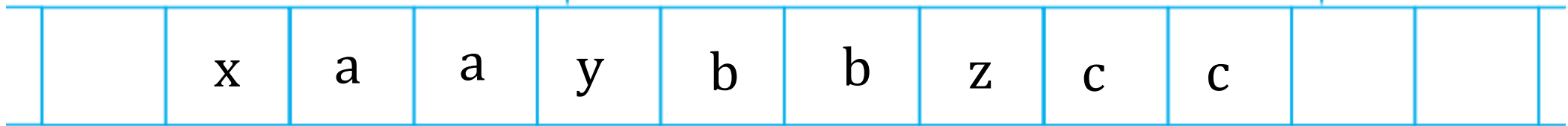
Example 9.8

- For $\Sigma = \{a, b, c\}$, design a Turing machine that accepts $L = \{a^n b^n c^n : n \geq 1\}$.



Example 9.8

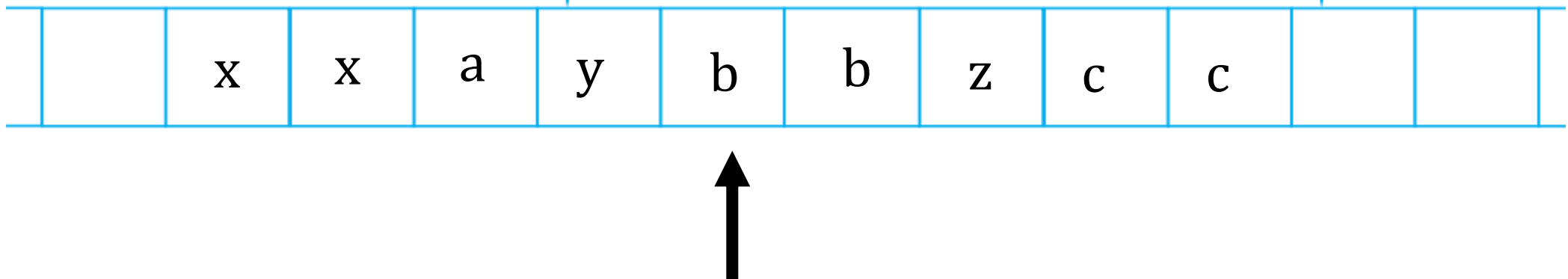
- For $\Sigma = \{a, b, c\}$, design a Turing machine that accepts $L = \{a^n b^n c^n : n \geq 1\}$.



Turing Machine

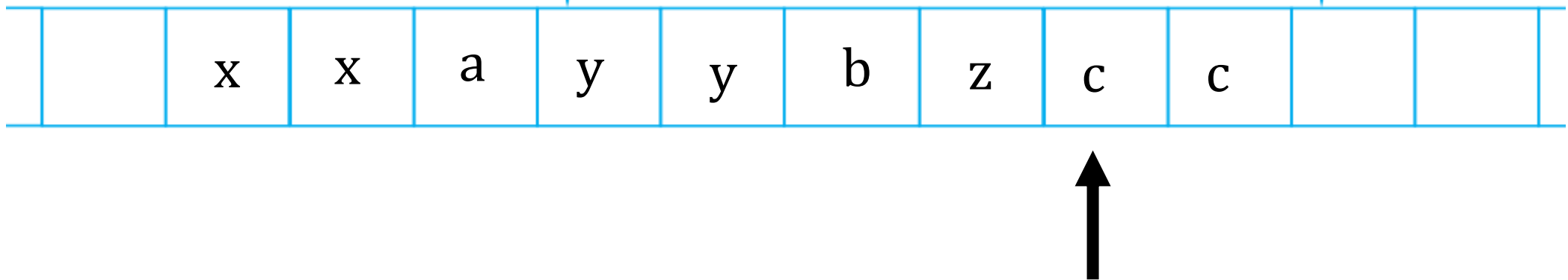
Example 9.8

- For $\Sigma = \{a, b, c\}$, design a Turing machine that accepts $L = \{a^n b^n c^n : n \geq 1\}$.



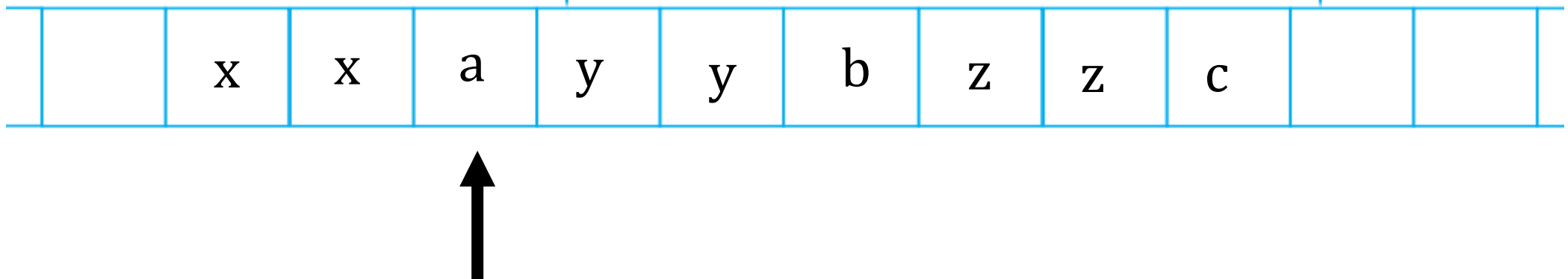
Example 9.8

- For $\Sigma = \{a, b, c\}$, design a Turing machine that accepts $L = \{a^n b^n c^n : n \geq 1\}$.



Example 9.8

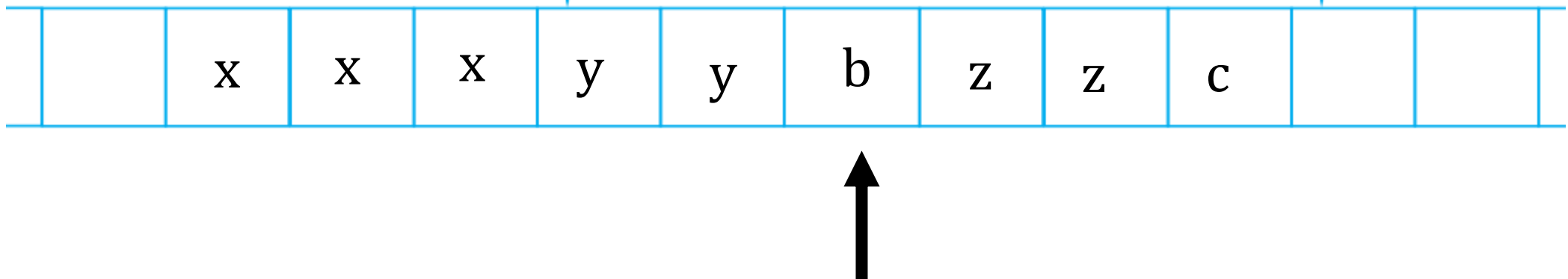
- For $\Sigma = \{a, b, c\}$, design a Turing machine that accepts $L = \{a^n b^n c^n : n \geq 1\}$.



Turing Machine

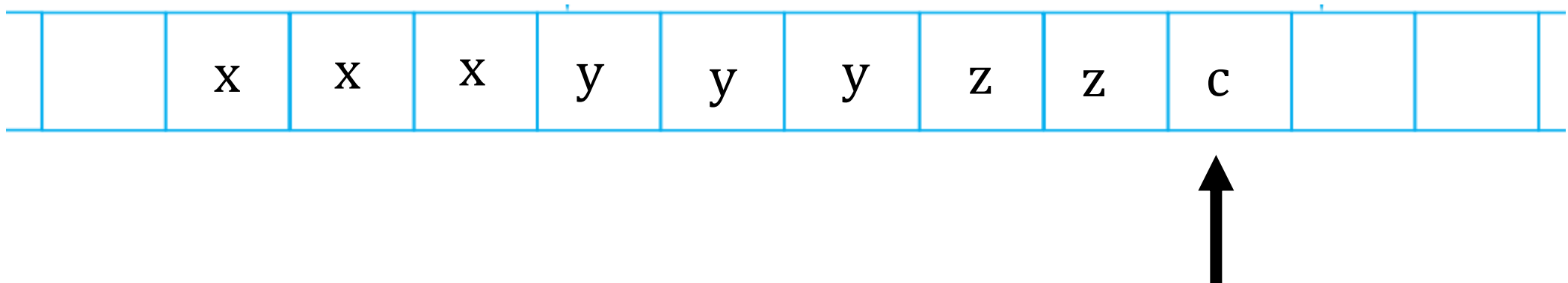
Example 9.8

- For $\Sigma = \{a, b, c\}$, design a Turing machine that accepts $L = \{a^n b^n c^n : n \geq 1\}$.



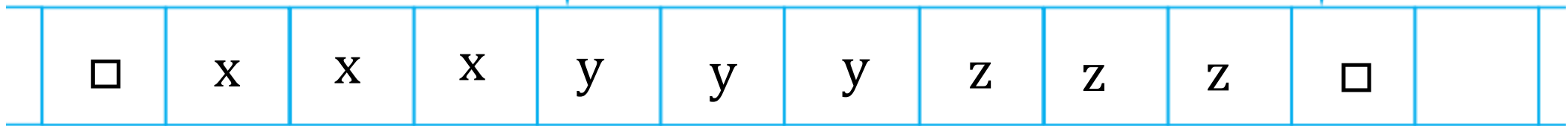
Example 9.8

- For $\Sigma = \{a, b, c\}$, design a Turing machine that accepts $L = \{a^n b^n c^n : n \geq 1\}$.



Example 9.8

- For $\Sigma = \{a, b, c\}$, design a Turing machine that accepts $L = \{a^n b^n c^n : n \geq 1\}$.




Turing Machine

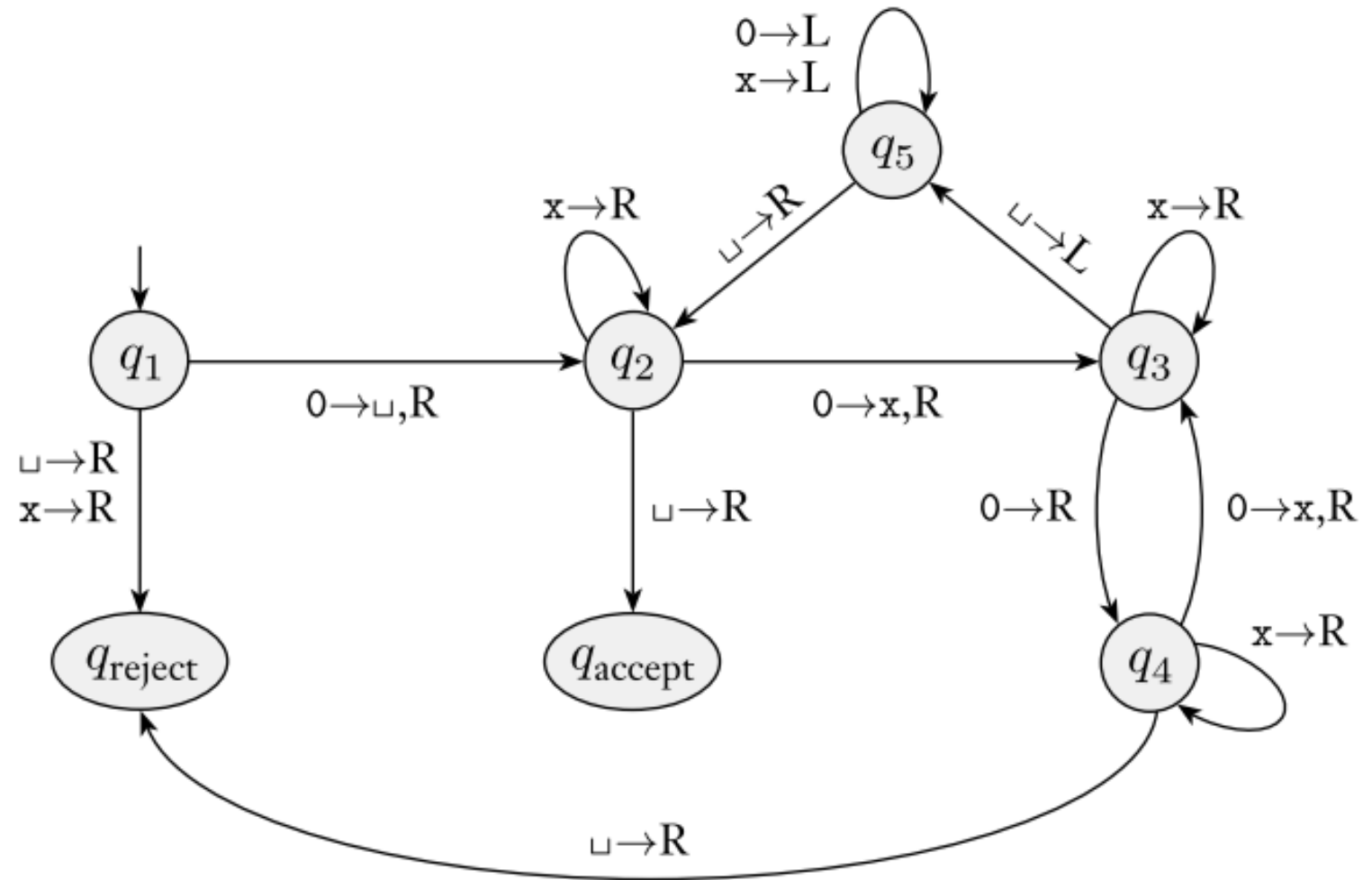
Example 3.7

Sipser

Example 3.7

- Here we describe a Turing machine (TM) M_2 that decides $A = \{0^{2^n} : n \geq 0\}$ the language consisting of all strings of 0s whose length is a power of 2.
 1. Sweep left to right across the tape, crossing off every other 0.
 2. If in stage 1 the tape contained a single 0, *accept*.
 3. If in stage 1 the tape contained more than a single 0 and the number of 0s was odd, *reject*.
 4. Return the head to the left-hand end of the tape.
 5. Go to stage 1.

Example 3.7

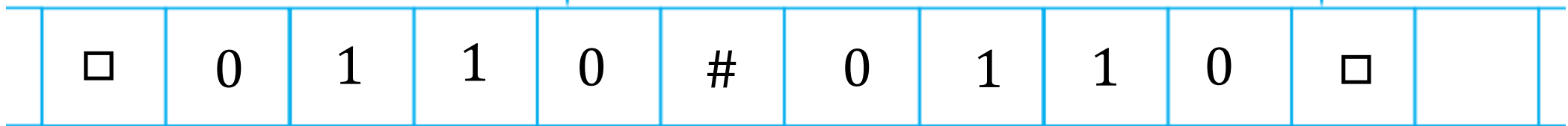


Example 3.9

Sipser

Example 3.9

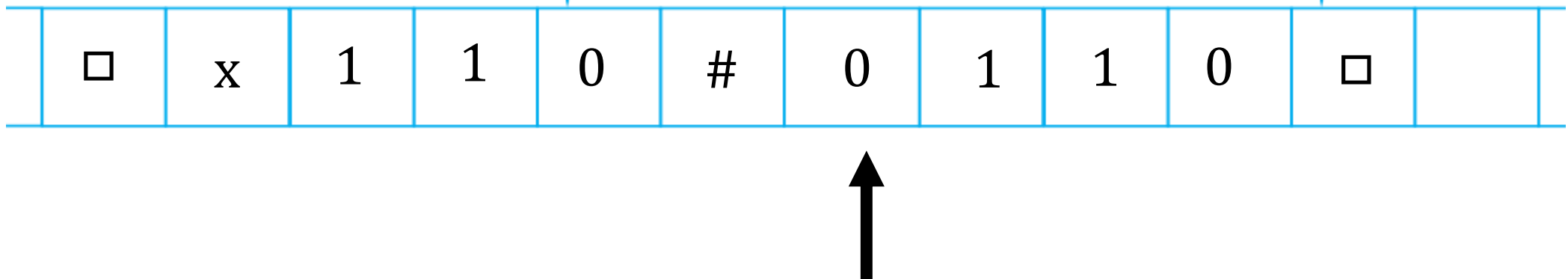
- The following is a formal description of $M_1 = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$, the Turing machine that we informally described for deciding the language $B = \{w\#w \mid w \in \{0,1\}^*\}$.



Turing Machine

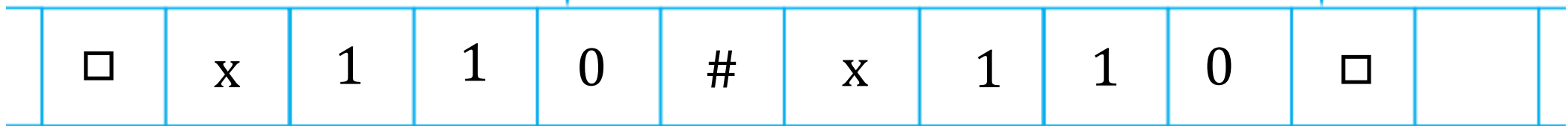
Example 3.9

- The following is a formal description of $M_1 = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$, the Turing machine that we informally described for deciding the language $B = \{w\#w \mid w \in \{0,1\}^*\}$.



Example 3.9

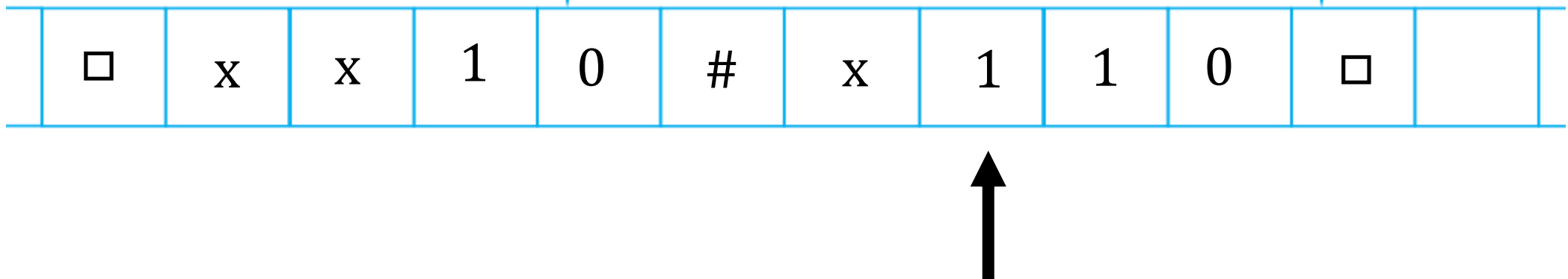
- The following is a formal description of $M_1 = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$, the Turing machine that we informally described for deciding the language $B = \{w\#w \mid w \in \{0,1\}^*\}$.



Turing Machine

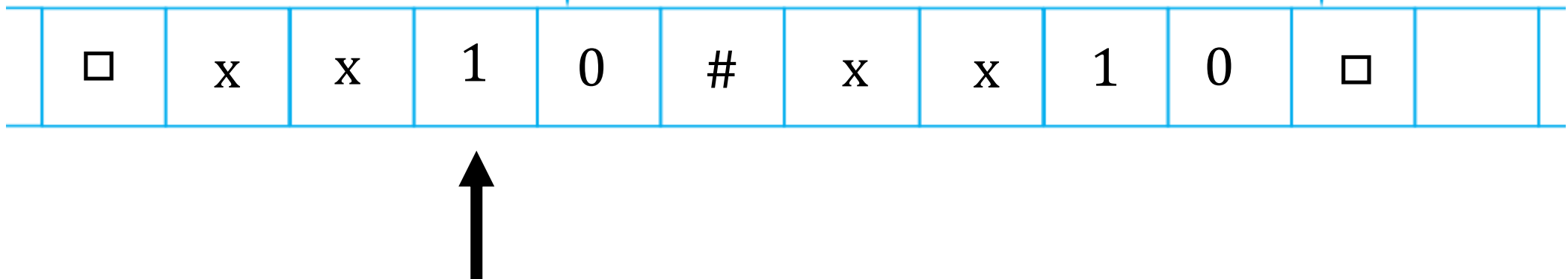
Example 3.9

- The following is a formal description of $M_1 = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$, the Turing machine that we informally described for deciding the language $B = \{w\#w \mid w \in \{0,1\}^*\}$.



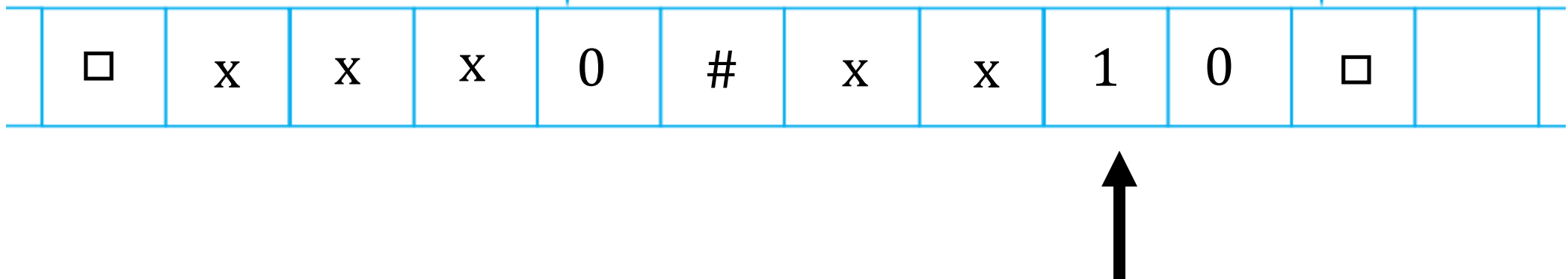
Example 3.9

- The following is a formal description of $M_1 = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$, the Turing machine that we informally described for deciding the language $B = \{w\#w \mid w \in \{0,1\}^*\}$.



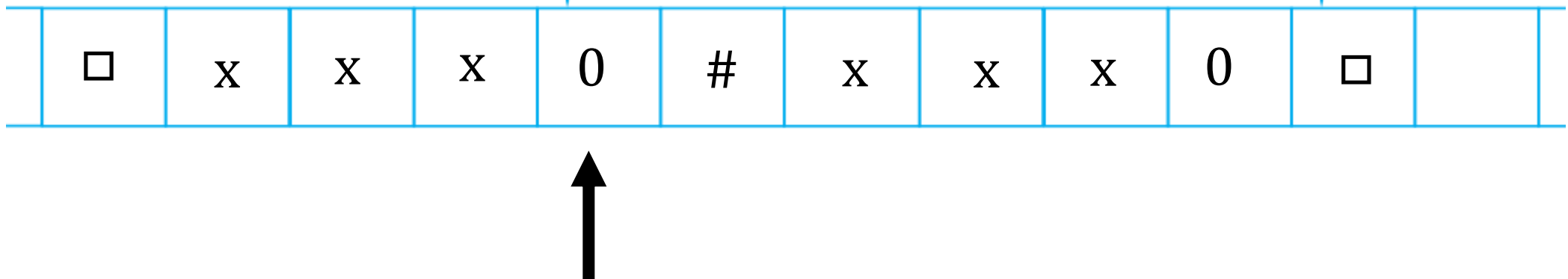
Example 3.9

- The following is a formal description of $M_1 = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$, the Turing machine that we informally described for deciding the language $B = \{w\#w \mid w \in \{0,1\}^*\}$.



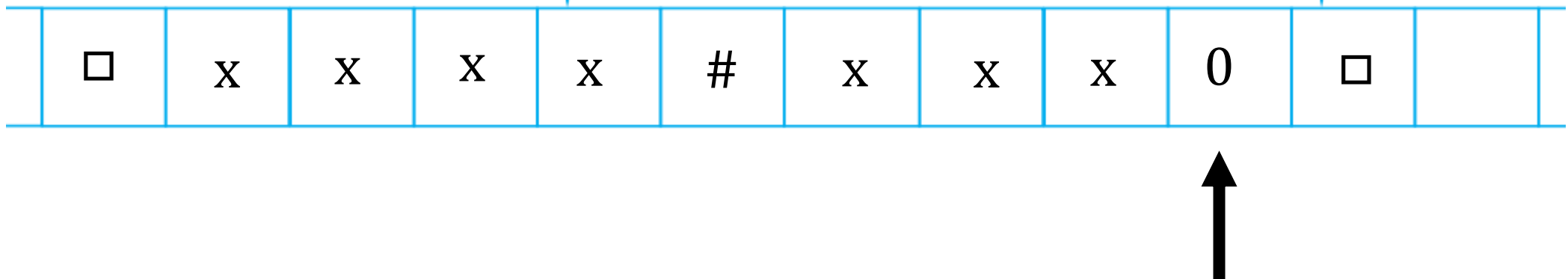
Example 3.9

- The following is a formal description of $M_1 = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$, the Turing machine that we informally described for deciding the language $B = \{w\#w \mid w \in \{0,1\}^*\}$.



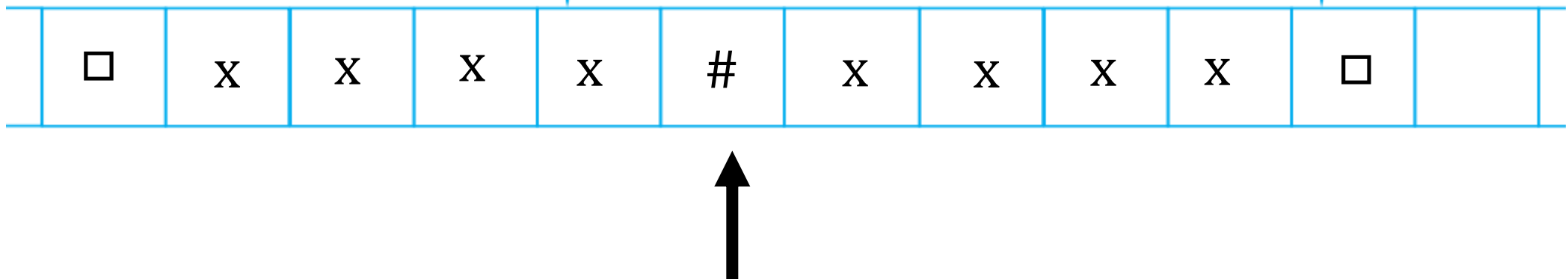
Example 3.9

- The following is a formal description of $M_1 = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$, the Turing machine that we informally described for deciding the language $B = \{w\#w \mid w \in \{0,1\}^*\}$.



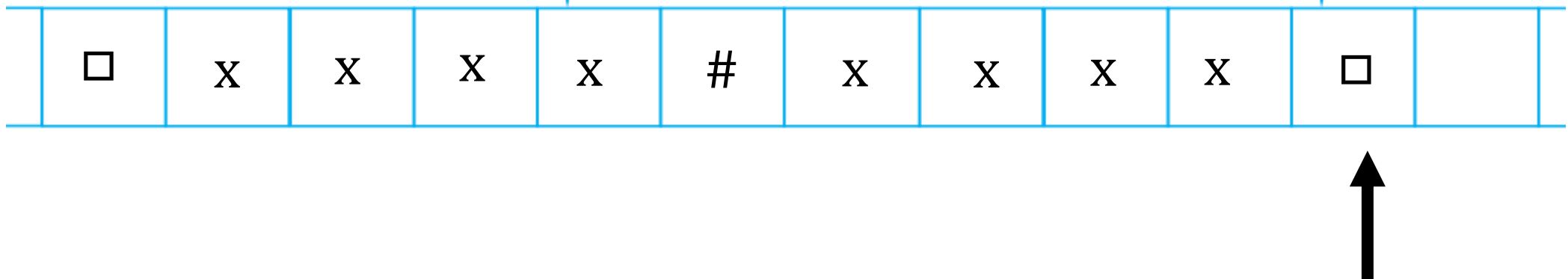
Example 3.9

- The following is a formal description of $M_1 = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$, the Turing machine that we informally described for deciding the language $B = \{w\#w \mid w \in \{0,1\}^*\}$.

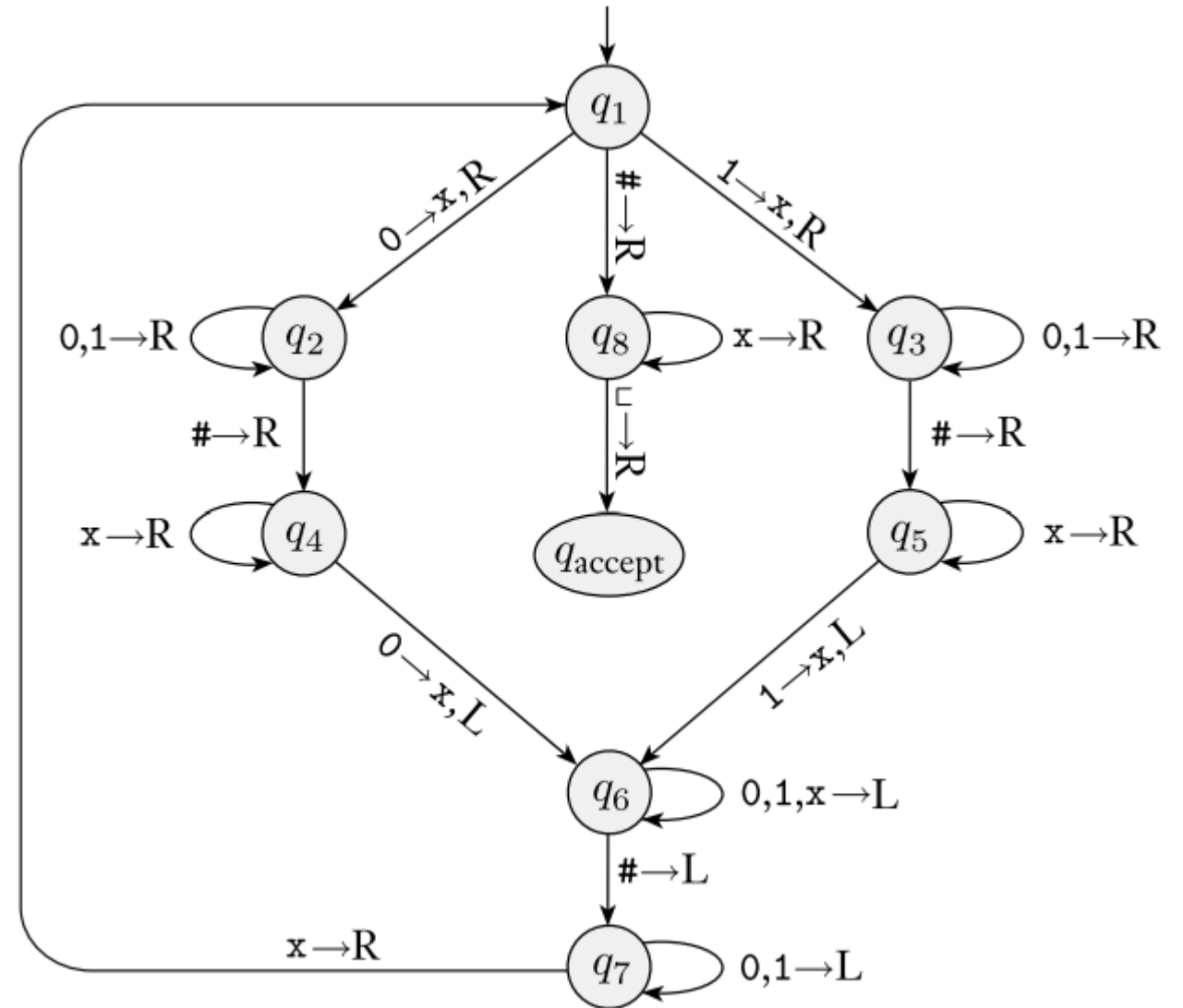


Example 3.9

- The following is a formal description of $M_1 = (Q, \Sigma, \Gamma, \delta, q_1, q_{\text{accept}}, q_{\text{reject}})$, the Turing machine that we informally described for deciding the language $B = \{w\#w \mid w \in \{0,1\}^*\}$.



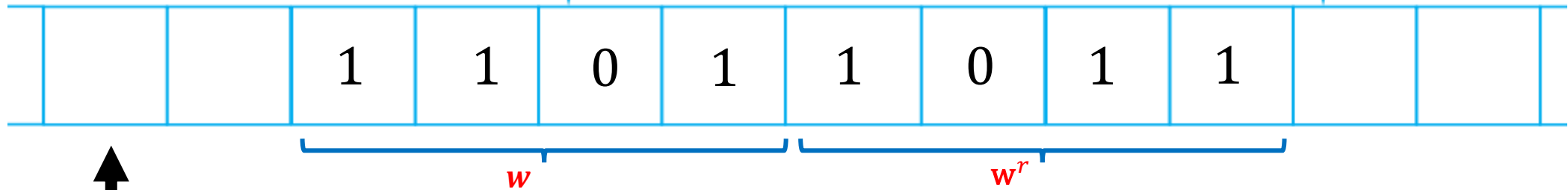
Example 3.9



Example 9.15

Example 9.15

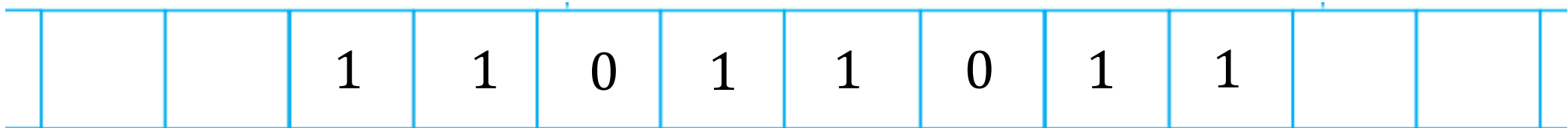
- $L = \{ww^r \mid w \in \{0,1\}^*\}$.



Turing Machine

Example 9.15

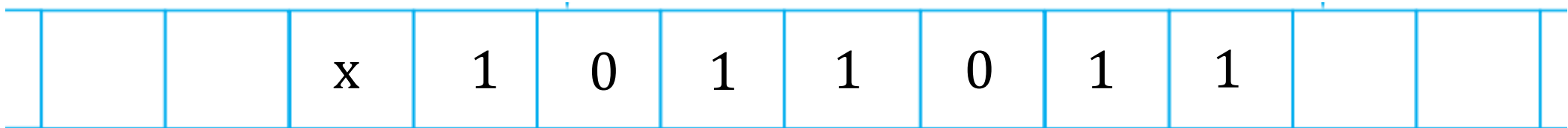
- $L = \{ww^r \mid w \in \{0,1\}^*\}$.



Turing Machine

Example 9.15

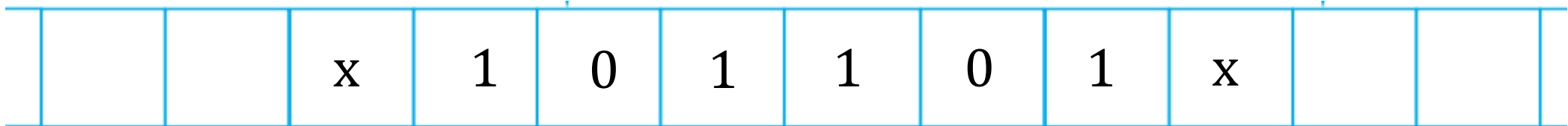
- $L = \{ww^r \mid w \in \{0,1\}^*\}$.



Turing Machine

Example 9.15

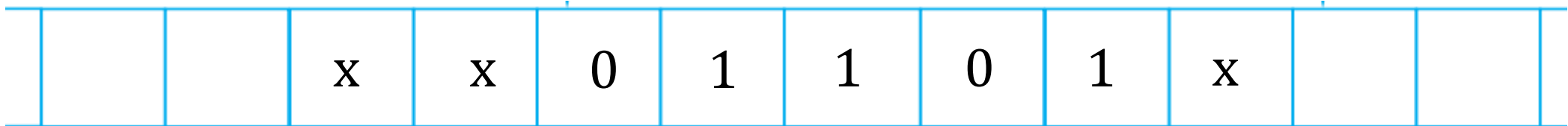
- $L = \{ww^r \mid w \in \{0,1\}^*\}$.



Turing Machine

Example 9.15

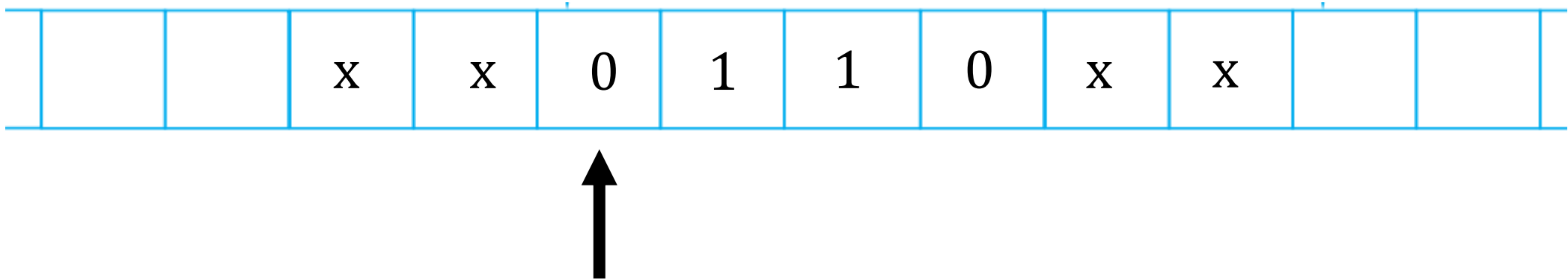
- $L = \{ww^r \mid w \in \{0,1\}^*\}$.



Turing Machine

Example 9.15

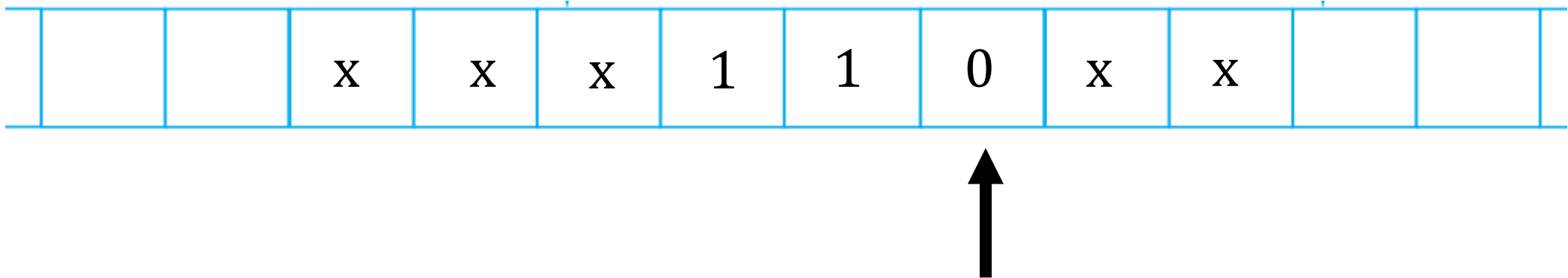
- $L = \{ww^r \mid w \in \{0,1\}^*\}$.



Turing Machine

Example 9.15

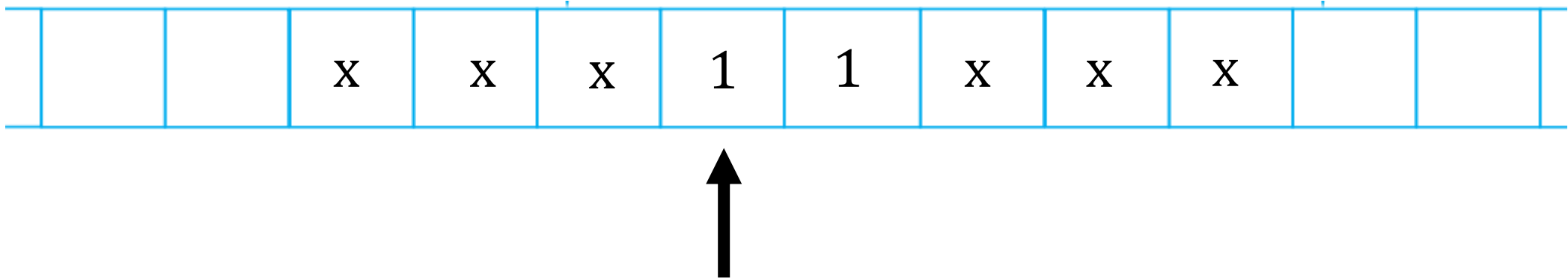
- $L = \{ww^r \mid w \in \{0,1\}^*\}$.



Turing Machine

Example 9.15

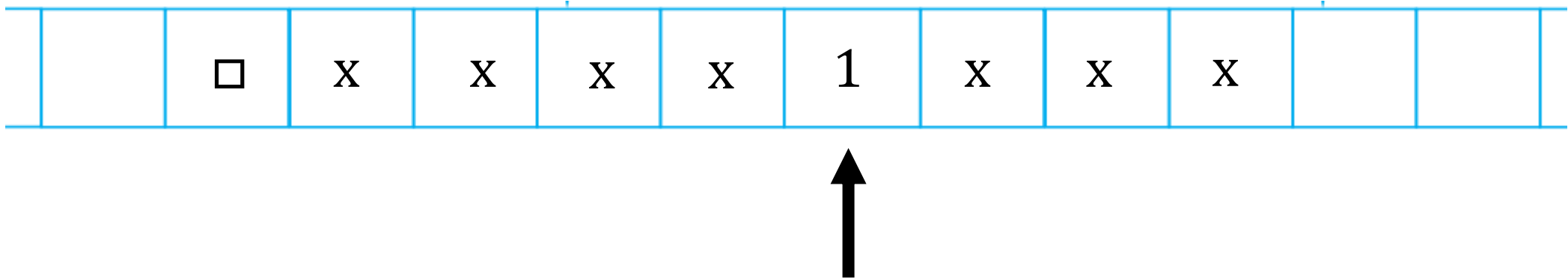
- $L = \{ww^r \mid w \in \{0,1\}^*\}$.



Turing Machine

Example 9.15

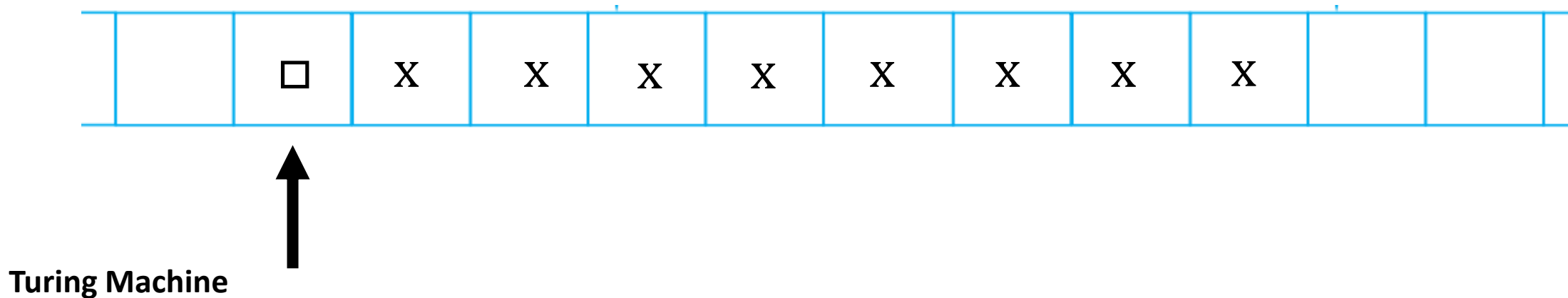
- $L = \{ww^r \mid w \in \{0,1\}^*\}$.



Turing Machine

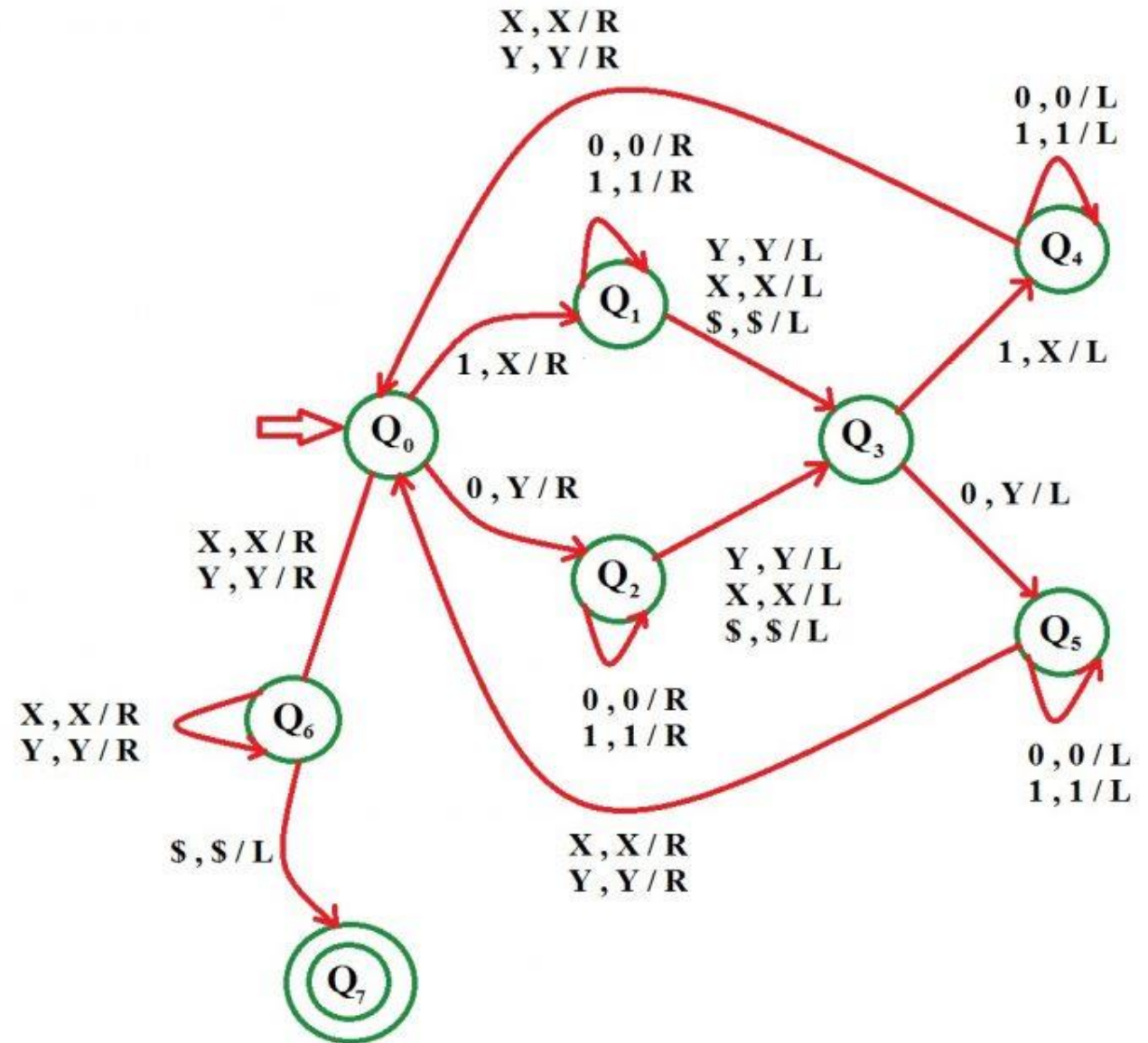
Example 9.15

- $L = \{ww^r \mid w \in \{0,1\}^*\}$.



Example 9.15

- $L = \{ww^r \mid w \in \{0,1\}^*\}$.

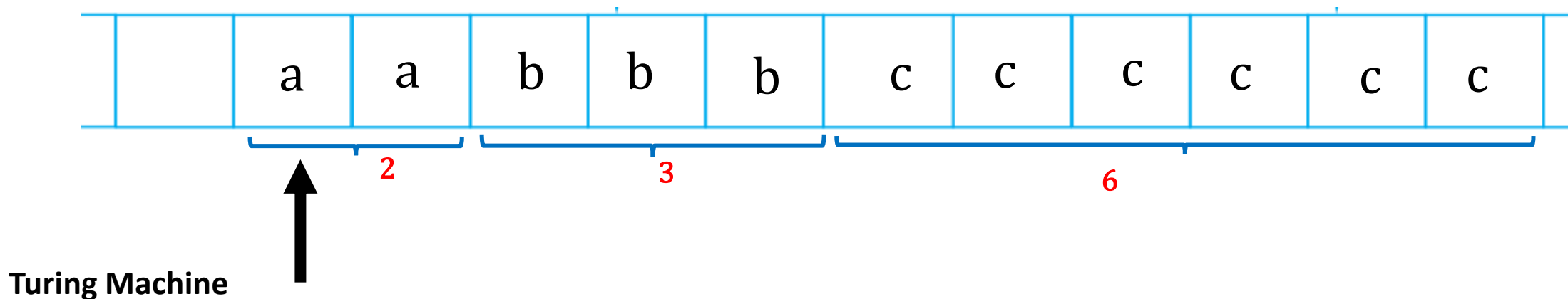


Examples 3.11

Sipser

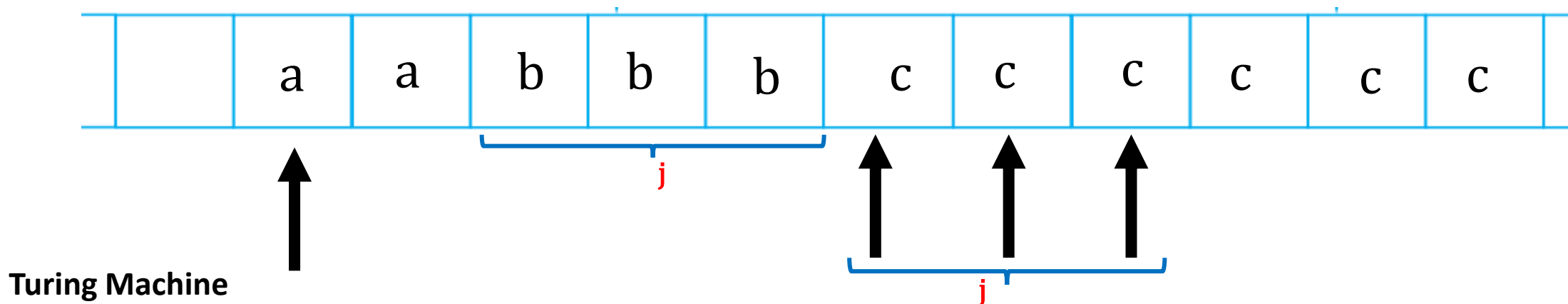
Example 3.11

- Here, a TM M_3 is doing some elementary arithmetic. It decides the language $C = \{a^i b^j c^k \mid i \times j = k \text{ and } i, j, k \geq 1\}$.



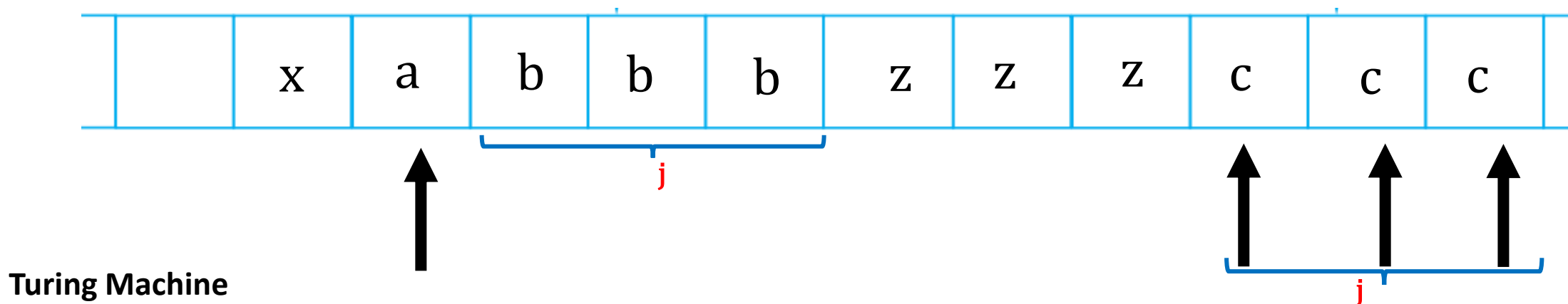
Example 3.11

- Here, a TM M_3 is doing some elementary arithmetic. It decides the language $C = \{a^i b^j c^k \mid i \times j = k \text{ and } i, j, k \geq 1\}$.



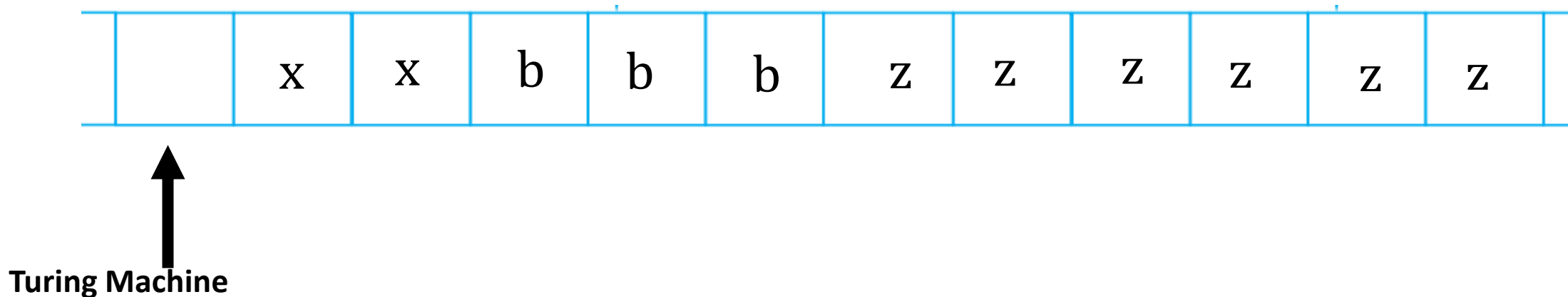
Example 3.11

- Here, a TM M_3 is doing some elementary arithmetic. It decides the language $C = \{a^i b^j c^k \mid i \times j = k \text{ and } i, j, k \geq 1\}$.

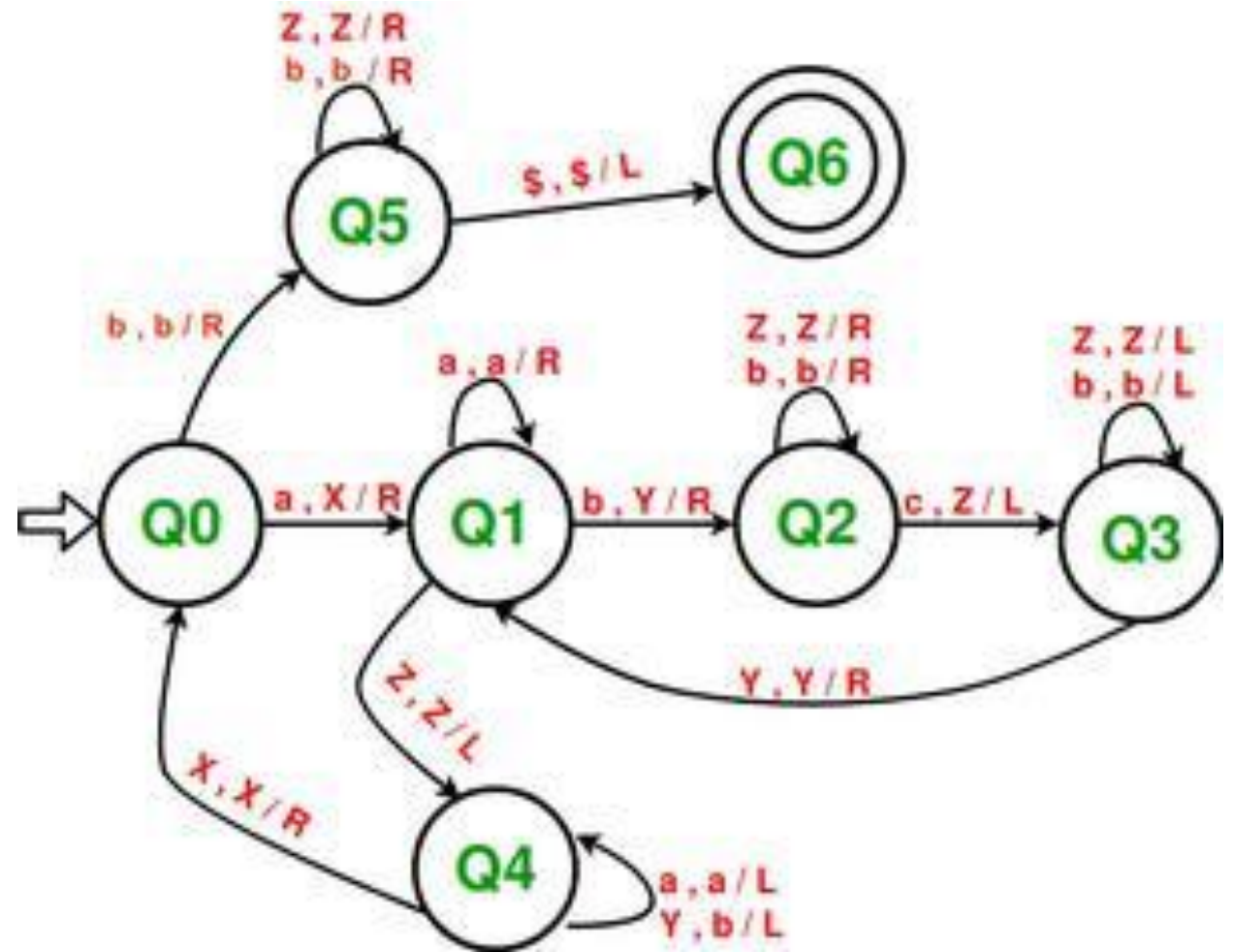


Example 3.11

- Here, a TM M_3 is doing some elementary arithmetic. It decides the language $C = \{a^i b^j c^k \mid i \times j = k \text{ and } i, j, k \geq 1\}$.



Example 3.11



Example 8.8

Kandar

Example 8.8

- design a Turing machine that accepts the string $L = (a,b)^*$, where $N(a) + N(b) = \text{even}$.

Solution: The string consists of any combination of 'a' and 'b'. In the string, the total number of characters is even. The TM is designed as follows. If the machine gets 'a' or 'b', it moves right with the state changed from q_0 to q_1 . In state q_1 , if the machine gets 'a' or 'b', it moves right with state changed from q_1 to q_0 . In state q_0 , if the machine gets a blank symbol, it halts.

The transitional functions are

$$\delta(q_0, a) \rightarrow (q_1, a, R)$$

$$\delta(q_0, b) \rightarrow (q_1, b, R)$$

$$\delta(q_1, a) \rightarrow (q_0, a, R)$$

$$\delta(q_1, b) \rightarrow (q_0, b, R)$$

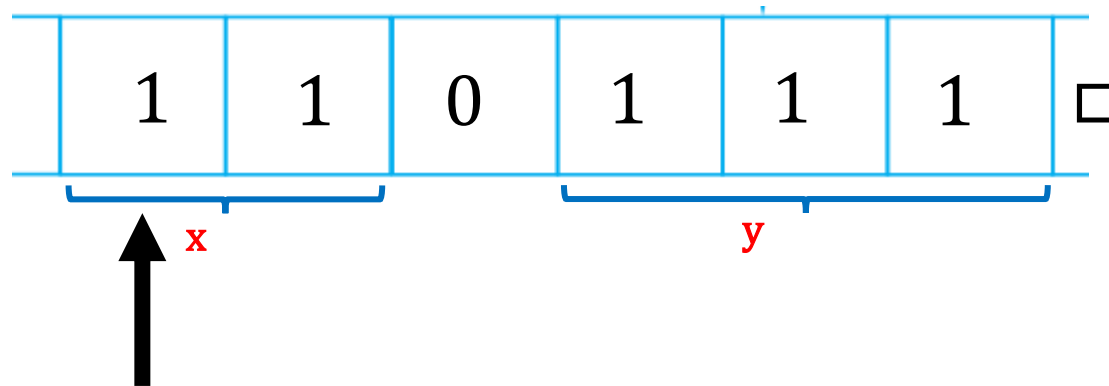
$$\delta(q_0, B) \rightarrow (q_f, B, H)$$

Turing Machine as Transducers

Example 9.9

Example 9.9

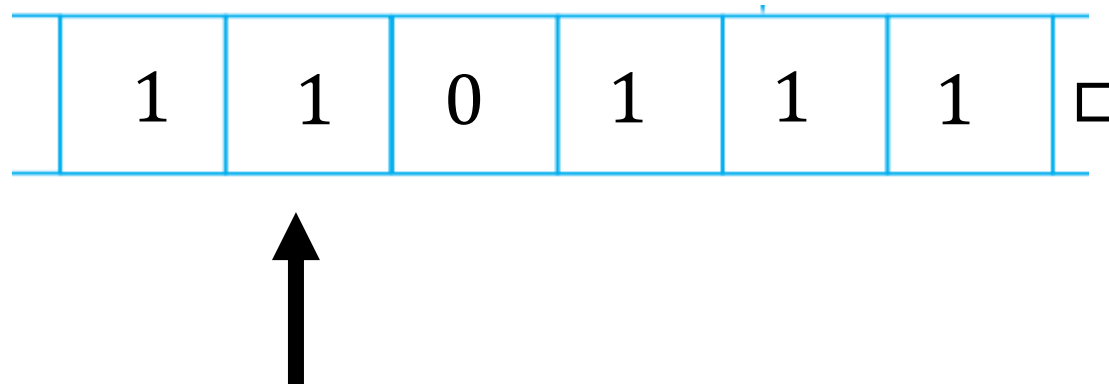
- Given two positive integers x and y , design a Turing machine that computes $x + y$.



Turing Machine

Example 9.9

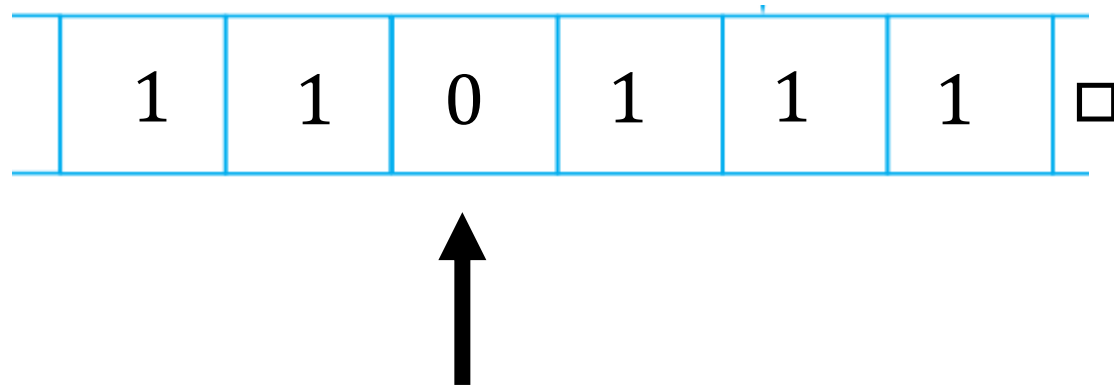
- Given two positive integers x and y , design a Turing machine that computes $x + y$.



Turing Machine

Example 9.9

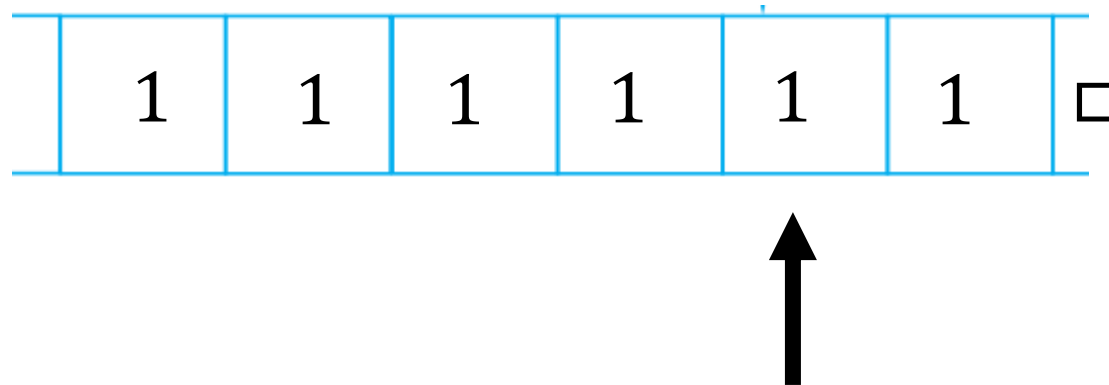
- Given two positive integers x and y , design a Turing machine that computes $x + y$.



Turing Machine

Example 9.9

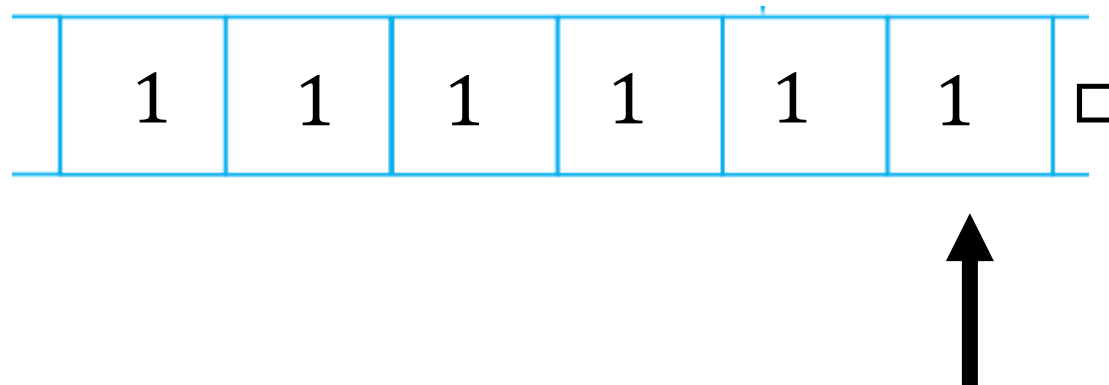
- Given two positive integers x and y , design a Turing machine that computes $x + y$.



Turing Machine

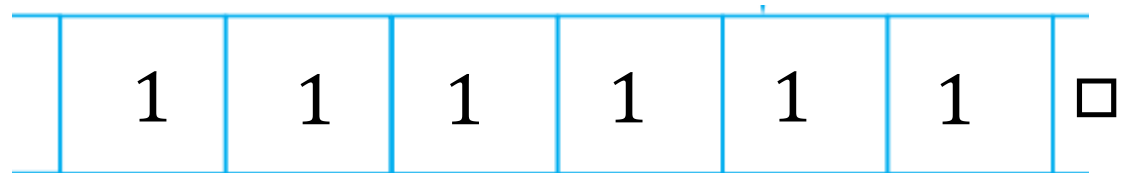
Example 9.9

- Given two positive integers x and y , design a Turing machine that computes $x + y$.



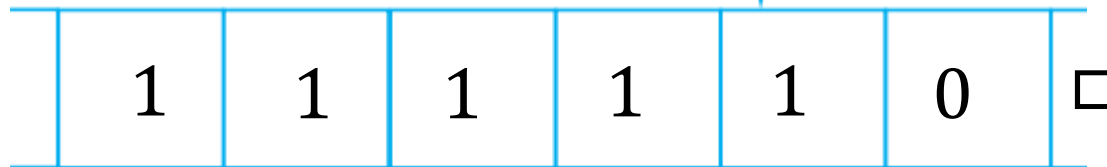
Example 9.9

- Given two positive integers x and y , design a Turing machine that computes $x + y$.



Example 9.9

- Given two positive integers x and y , design a Turing machine that computes $x + y$.

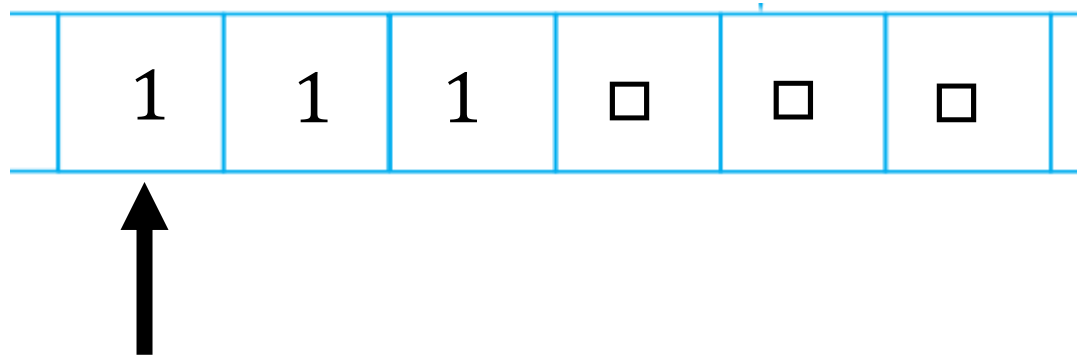


$$\begin{aligned}\delta(q_0, 1) &= (q_0, 1, R), \\ \delta(q_0, 0) &= (q_1, 1, R), \\ \delta(q_1, 1) &= (q_1, 1, R), \\ \delta(q_1, \square) &= (q_2, \square, L), \\ \delta(q_2, 1) &= (q_3, 0, L), \\ \delta(q_3, 1) &= (q_3, 1, L), \\ \delta(q_3, \square) &= (q_4, \square, R).\end{aligned}$$

Example 9.10

Example 9.10

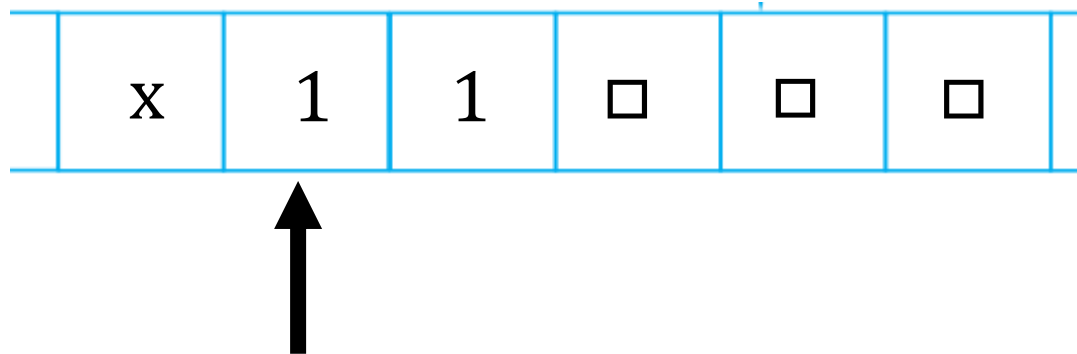
- Design a Turing machine that copies strings of 1's. More precisely, find a machine that performs the computation $q_0w \vdash^* q_f ww$, for any $w \in \{1\}^+$.



Turing Machine

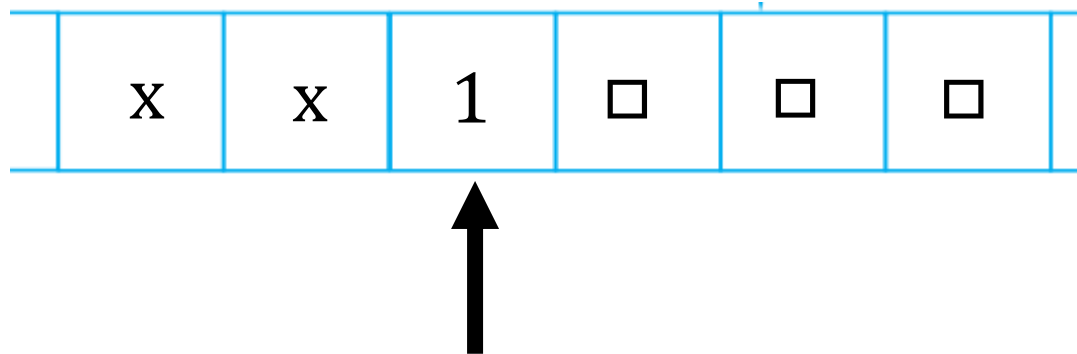
Example 9.10

- Design a Turing machine that copies strings of 1's. More precisely, find a machine that performs the computation $q_0w \vdash^* q_f ww$, for any $w \in \{1\}^+$.



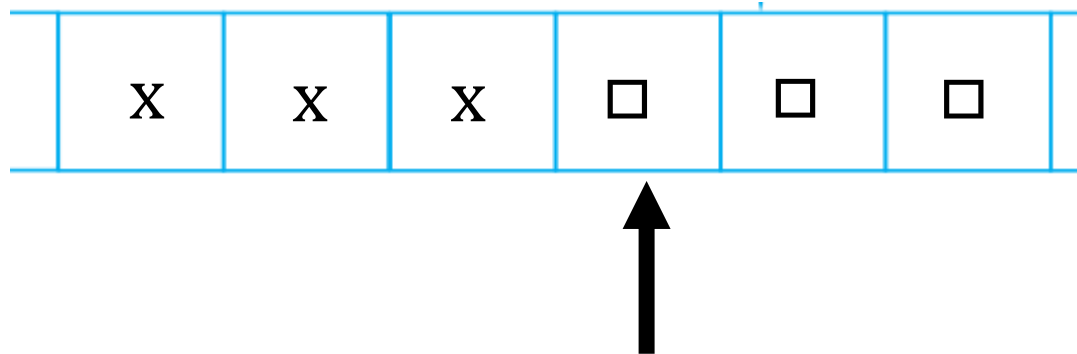
Example 9.10

- Design a Turing machine that copies strings of 1's. More precisely, find a machine that performs the computation $q_0w \vdash^* q_f ww$, for any $w \in \{1\}^+$.



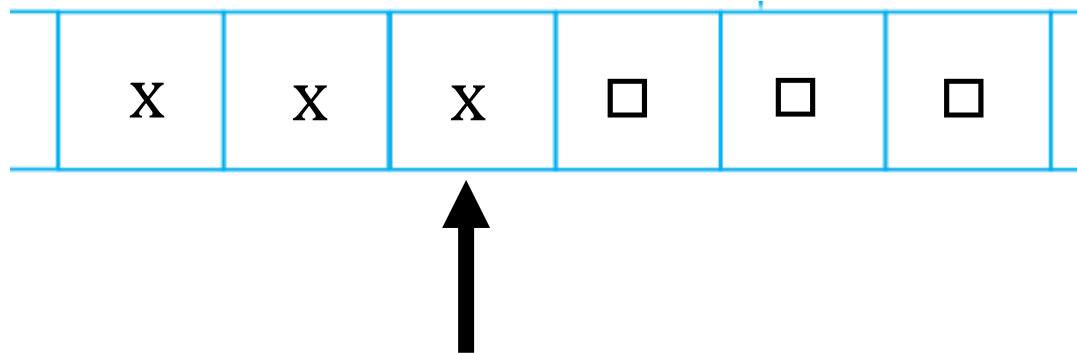
Example 9.10

- Design a Turing machine that copies strings of 1's. More precisely, find a machine that performs the computation $q_0w \vdash^* q_fw w$, for any $w \in \{1\}^+$.



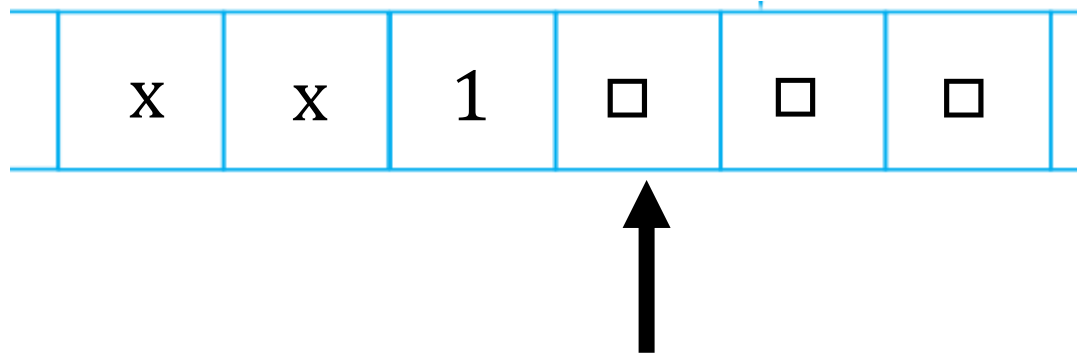
Example 9.10

- Design a Turing machine that copies strings of 1's. More precisely, find a machine that performs the computation $q_0w \vdash^* q_f ww$, for any $w \in \{1\}^+$.



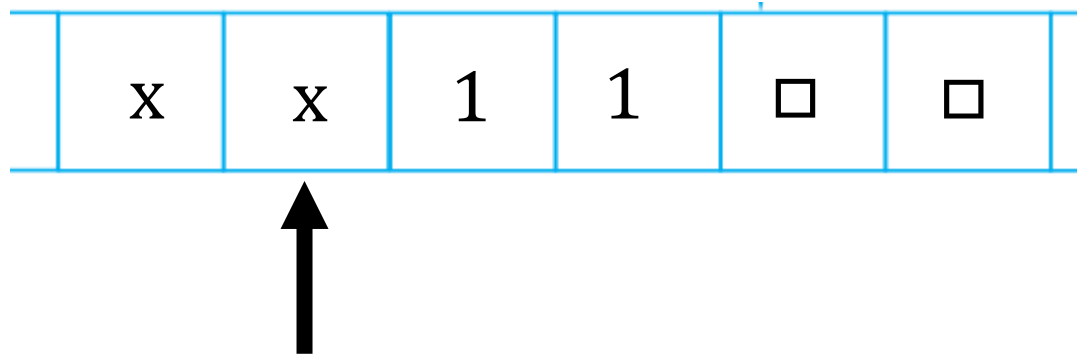
Example 9.10

- Design a Turing machine that copies strings of 1's. More precisely, find a machine that performs the computation $q_0w \vdash^* q_f ww$, for any $w \in \{1\}^+$.



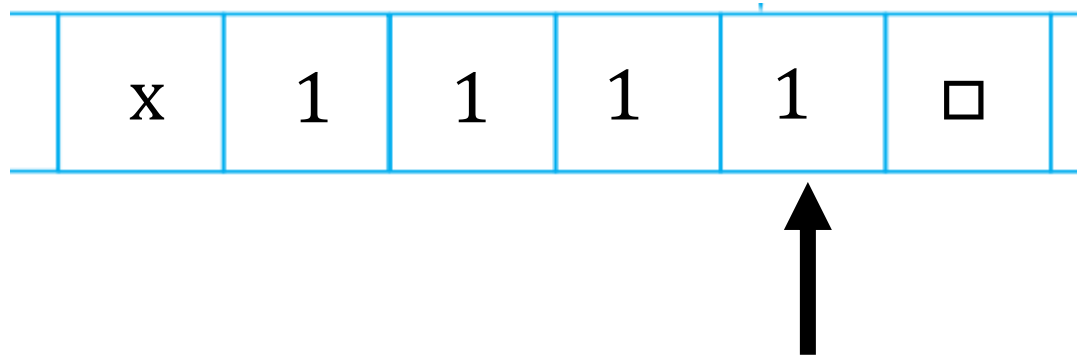
Example 9.10

- Design a Turing machine that copies strings of 1's. More precisely, find a machine that performs the computation $q_0w \vdash^* q_f ww$, for any $w \in \{1\}^+$.



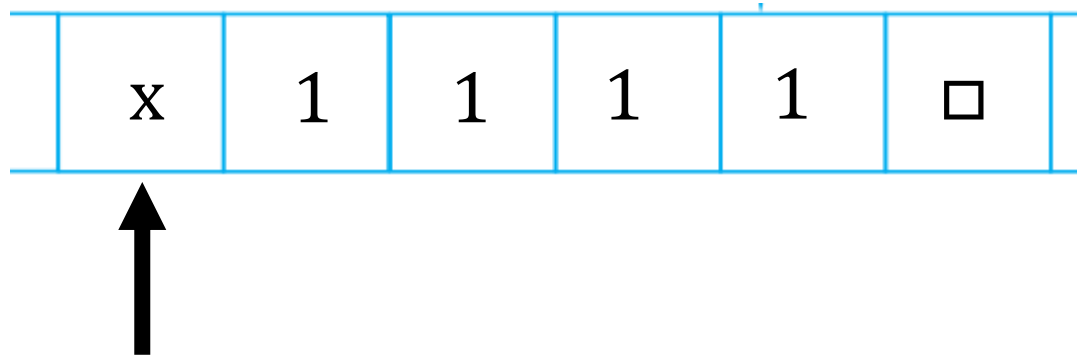
Example 9.10

- Design a Turing machine that copies strings of 1's. More precisely, find a machine that performs the computation $q_0w \vdash^* q_f ww$, for any $w \in \{1\}^+$.



Example 9.10

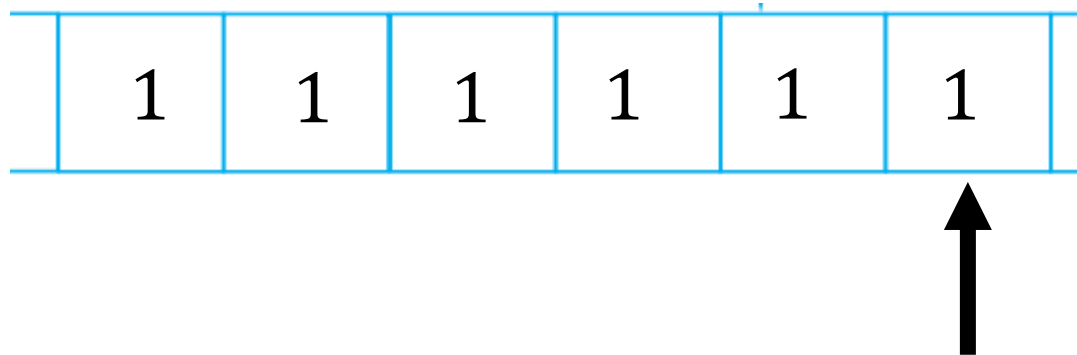
- Design a Turing machine that copies strings of 1's. More precisely, find a machine that performs the computation $q_0w \vdash^* q_f ww$, for any $w \in \{1\}^+$.



Turing Machine

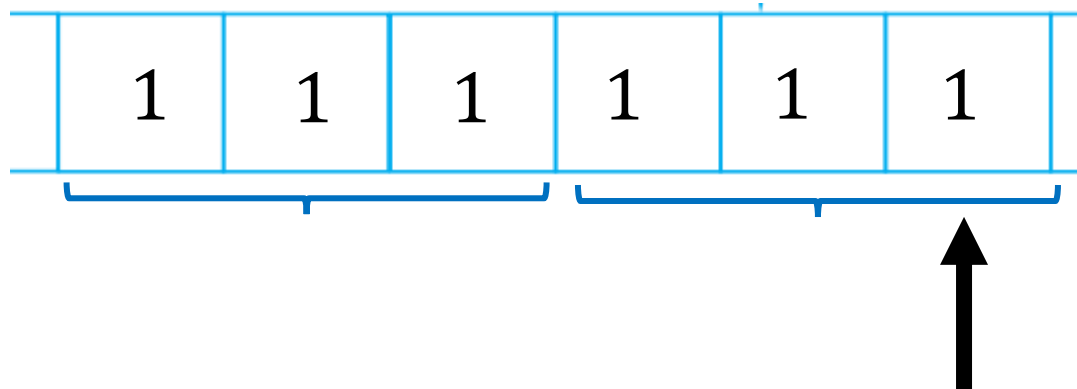
Example 9.10

- Design a Turing machine that copies strings of 1's. More precisely, find a machine that performs the computation $q_0w \vdash^* q_f ww$, for any $w \in \{1\}^+$.



Example 9.10

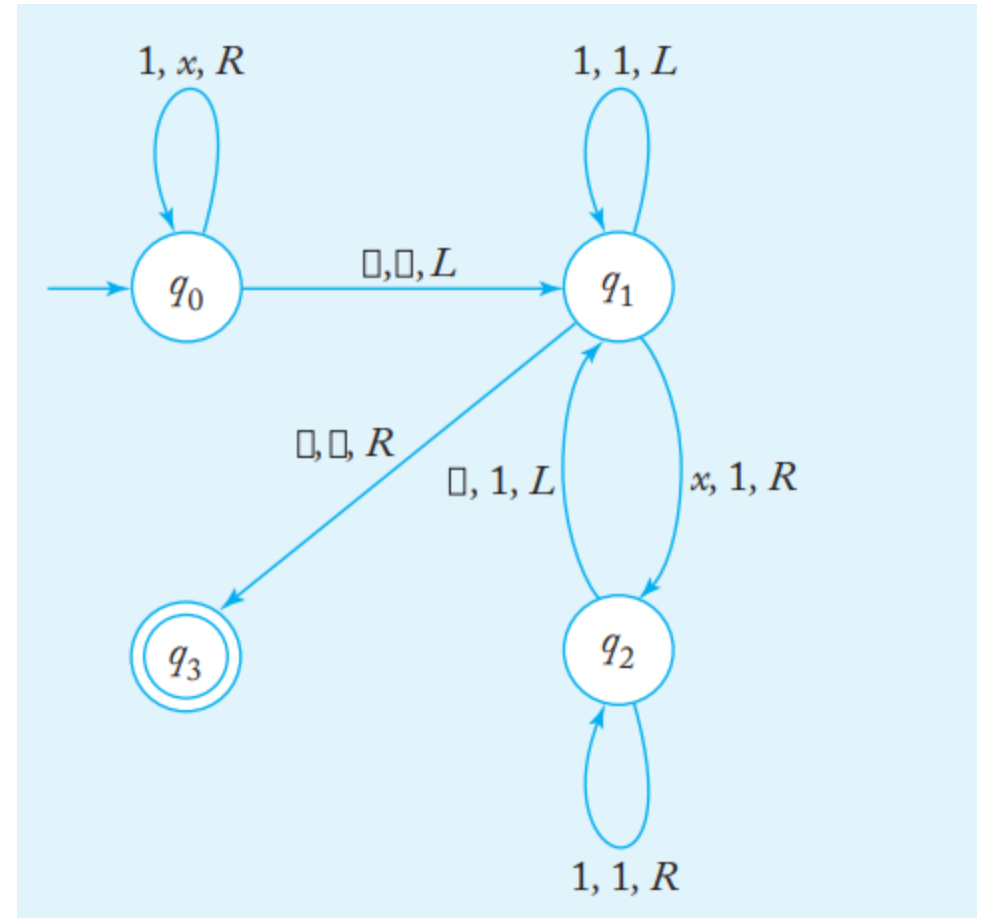
- Design a Turing machine that copies strings of 1's. More precisely, find a machine that performs the computation $q_0w \vdash^* q_f ww$, for any $w \in \{1\}^+$.



Turing Machine



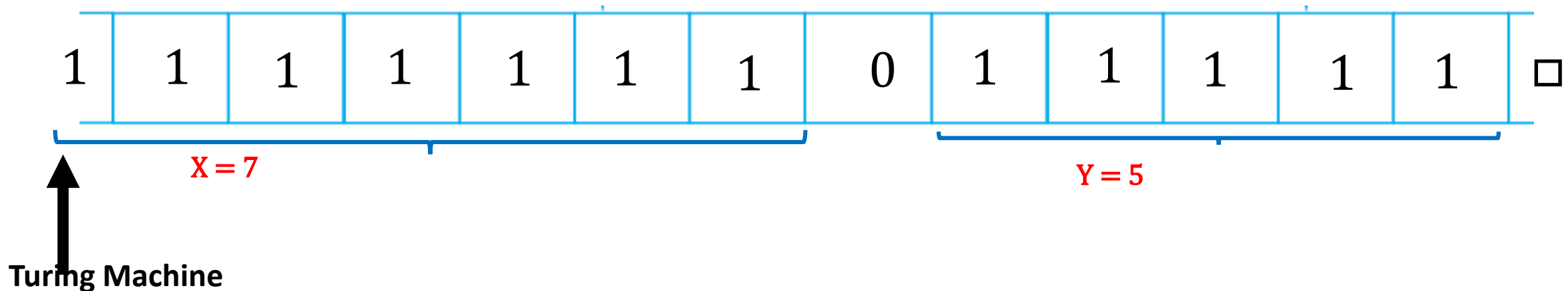
Example 9.10



Example 9.11

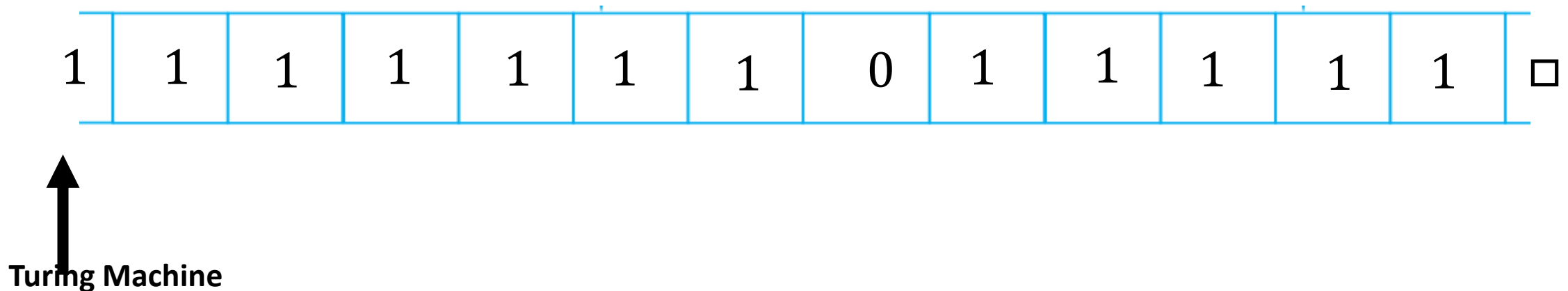
Example 9.11

- Design a Turing machine that compare x , y and if $x > y$ halt in final state and else $x < y$ halt in nonfinal state.



Example 9.11

- Design a Turing machine that compare x , y and if $x > y$ halt in final state and else $x < y$ halt in nonfinal state.



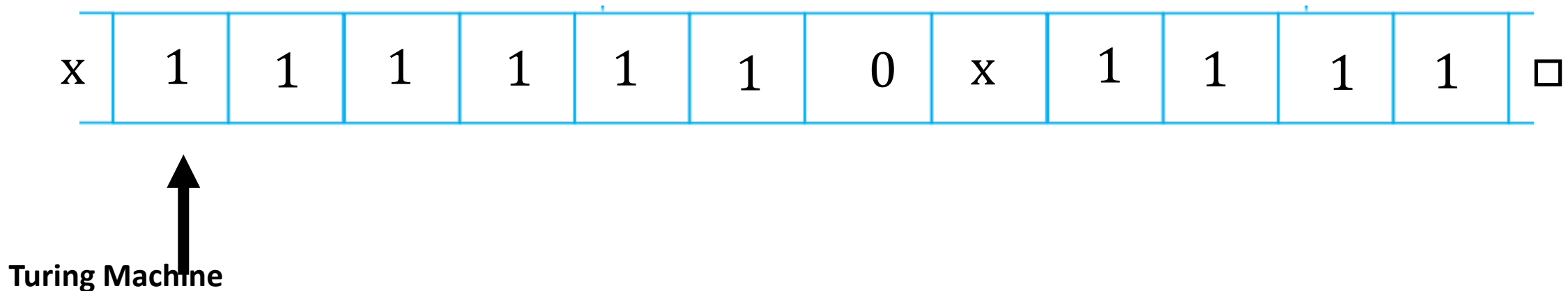
Example 9.11

- Design a Turing machine that compare x , y and if $x > y$ halt in final state and else $x < y$ halt in nonfinal state.



Example 9.11

- Design a Turing machine that compare x , y and if $x > y$ halt in final state and else $x < y$ halt in nonfinal state.



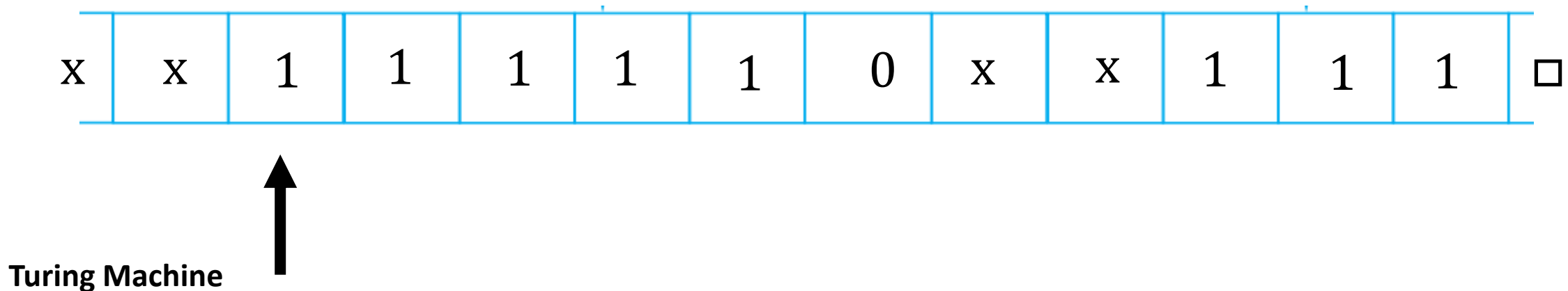
Example 9.11

- Design a Turing machine that compare x , y and if $x > y$ halt in final state and else $x < y$ halt in nonfinal state.



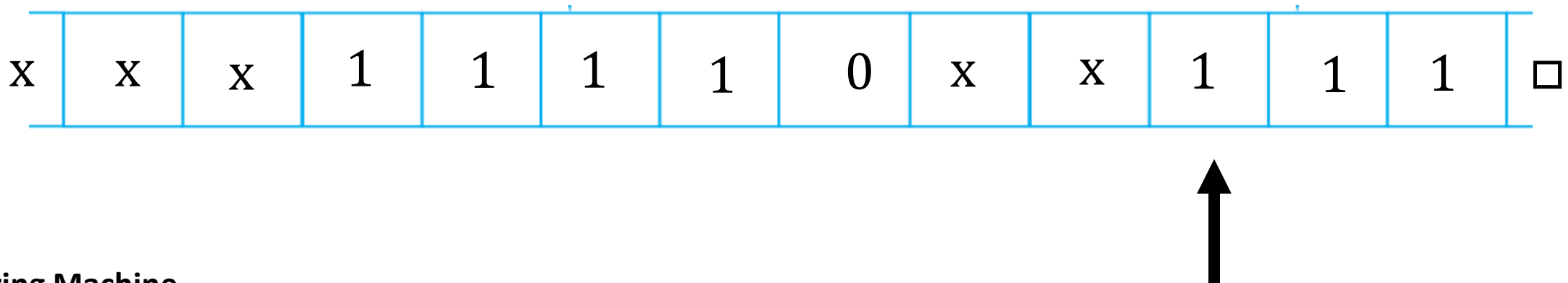
Example 9.11

- Design a Turing machine that compare x , y and if $x > y$ halt in final state and else $x < y$ halt in nonfinal state.



Example 9.11

- Design a Turing machine that compare x , y and if $x > y$ halt in final state and else $x < y$ halt in nonfinal state.



Turing Machine

Example 9.11

- Design a Turing machine that compare x , y and if $x > y$ halt in final state and else $x < y$ halt in nonfinal state.



Turing Machine

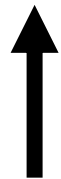
Example 9.11

- Design a Turing machine that compare x , y and if $x > y$ halt in final state and else $x < y$ halt in nonfinal state.



Example 9.11

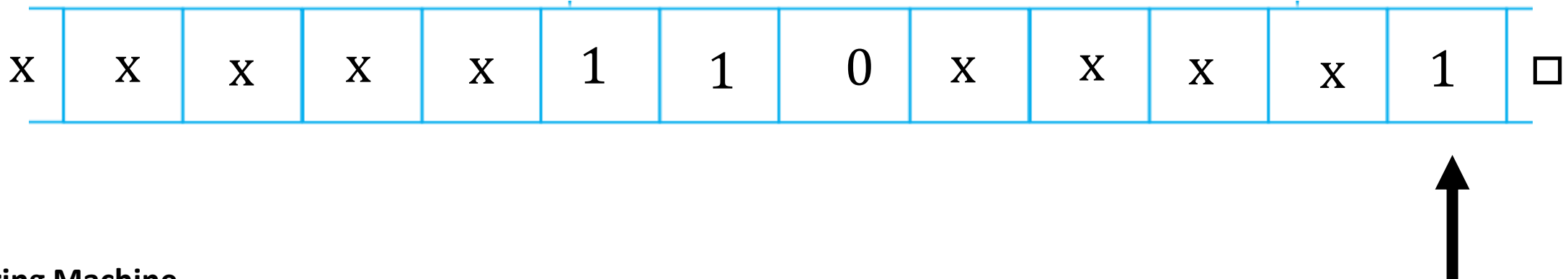
- Design a Turing machine that compare x , y and if $x > y$ halt in final state and else $x < y$ halt in nonfinal state.



Turing Machine

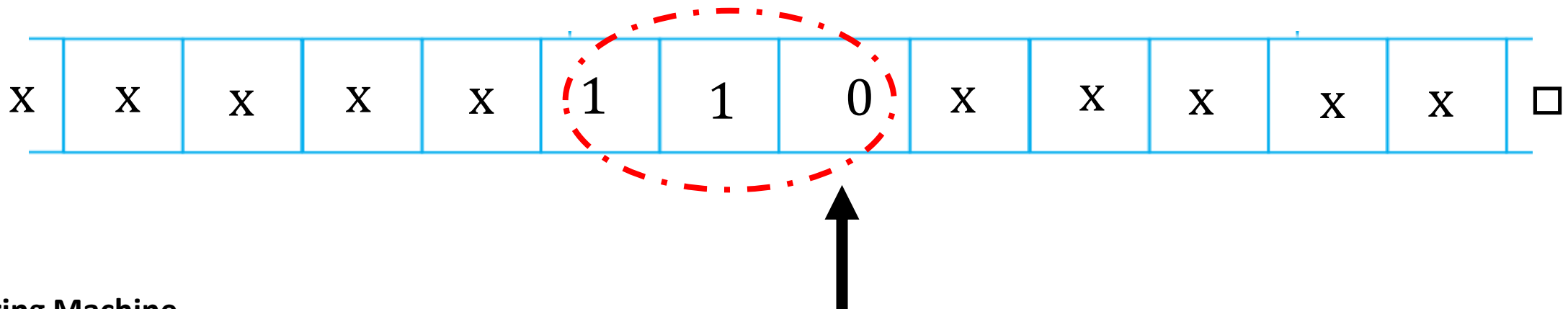
Example 9.11

- Design a Turing machine that compare x , y and if $x > y$ halt in final state and else $x < y$ halt in nonfinal state.



Example 9.11

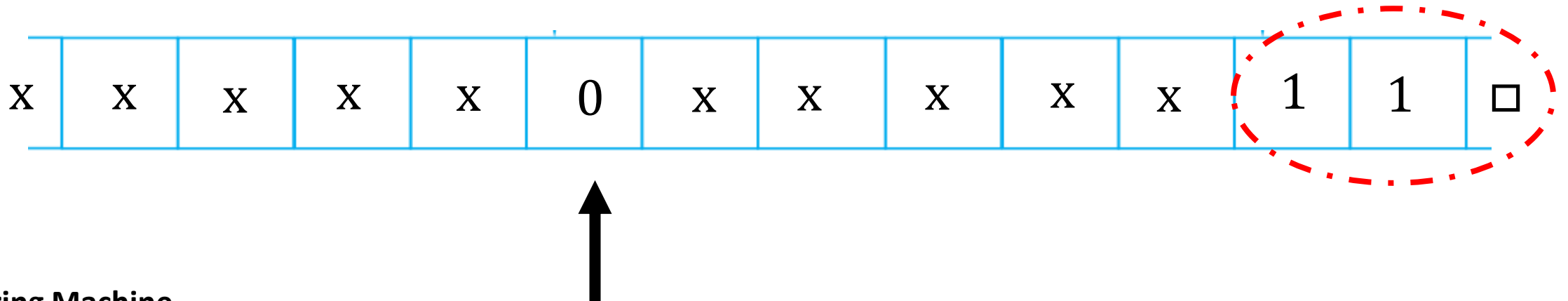
- Design a Turing machine that compare x , y and if $x > y$ halt in final state and else $x < y$ halt in nonfinal state.



Turing Machine

Example 9.11

- Design a Turing machine that compare x , y and if $x > y$ halt in final state and else $x < y$ halt in nonfinal state.



Example 6.1

Sunitha

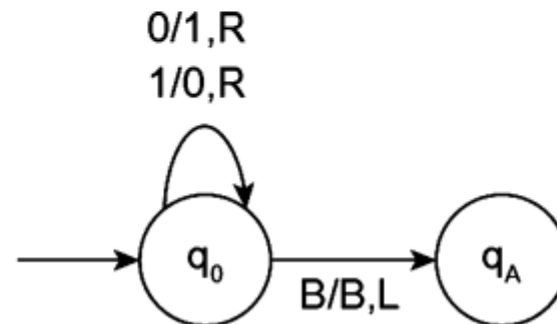
Example 6.1

- Design a Turing machine for Finding 1's complement of a given Binary number.

Solution: If the given number is 100110, then its 1's complement is 011001.

If the given number is 11110, then its 1's complement is 00001.

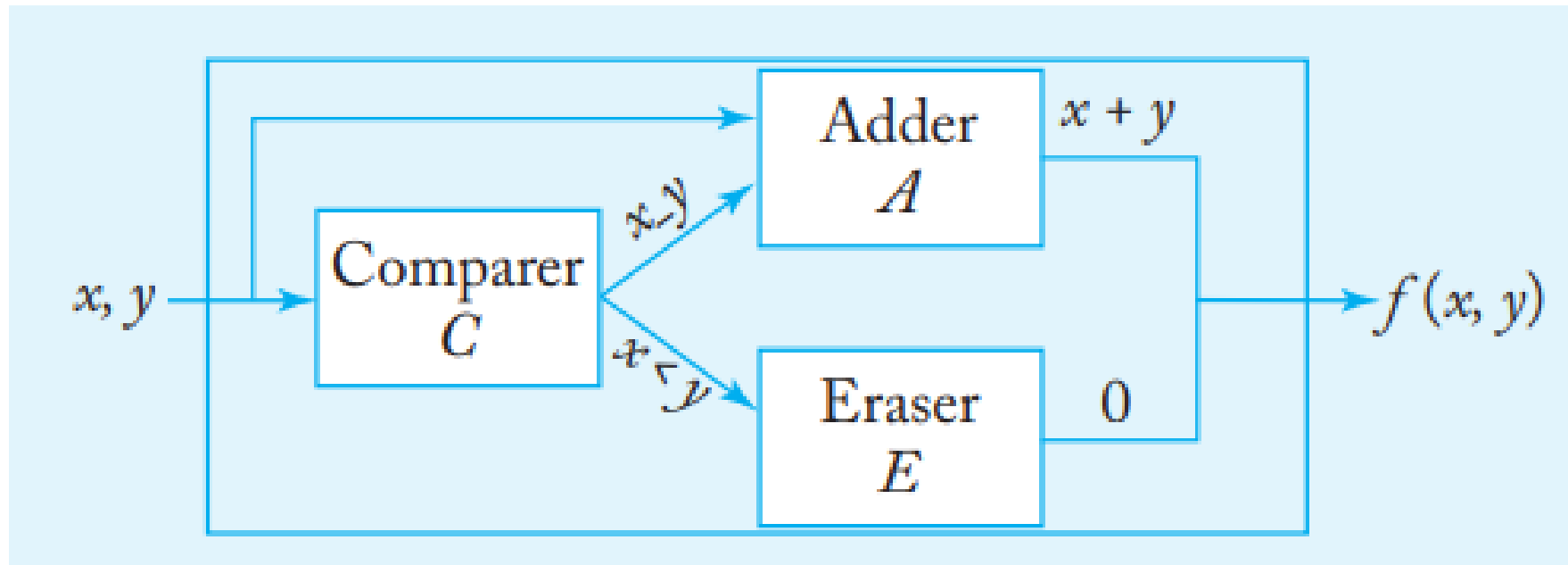
From the above examples, it is clear that the TM has to be designed such that while moving to right, if the input is 1, it should be changed to 0 and if the input is 0 it should be changed to 1 and halt on B. The transition diagram is shown in the Figure 6.2.



Example 9.12

Example 9.12

- Design a Turing machine that computes the function
$$\begin{cases} f(x, y) = x + y & ; \text{ if } x \geq y \\ = 0 & ; \text{ if } x < y \end{cases}$$



Example 9.13

Example 9.13

- If a then q_j else q_k

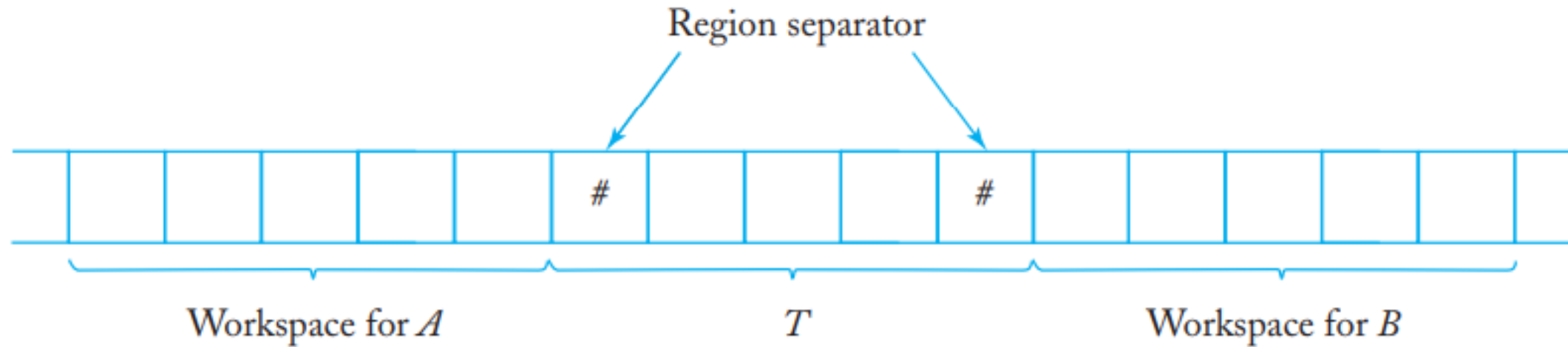
$$\delta(q_i, a) = (q_{j0}, a, R),$$

$$\delta(q_i, b) = (q_{k0}, b, R),$$

$$\delta(q_{j0}, c) = (q_j, c, L),$$

$$\delta(q_{k0}, c) = (q_k, c, L)$$

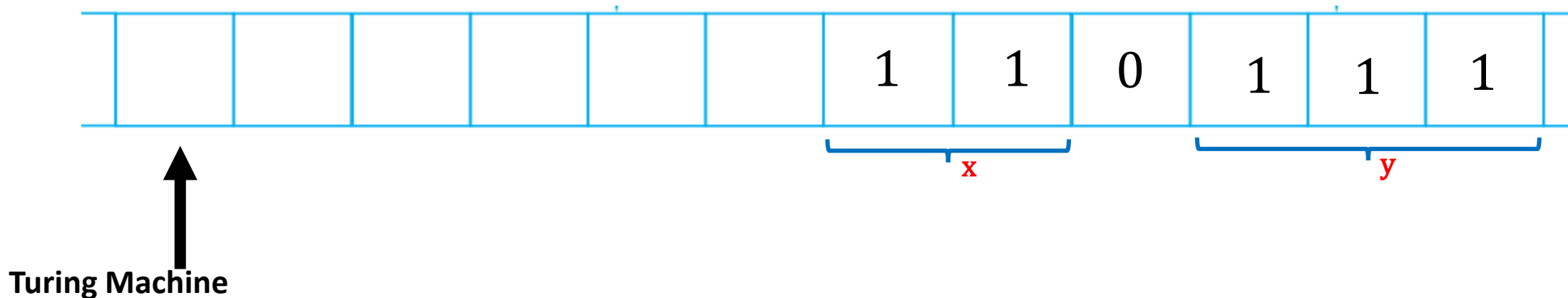
Example 9.13



Example 9.14

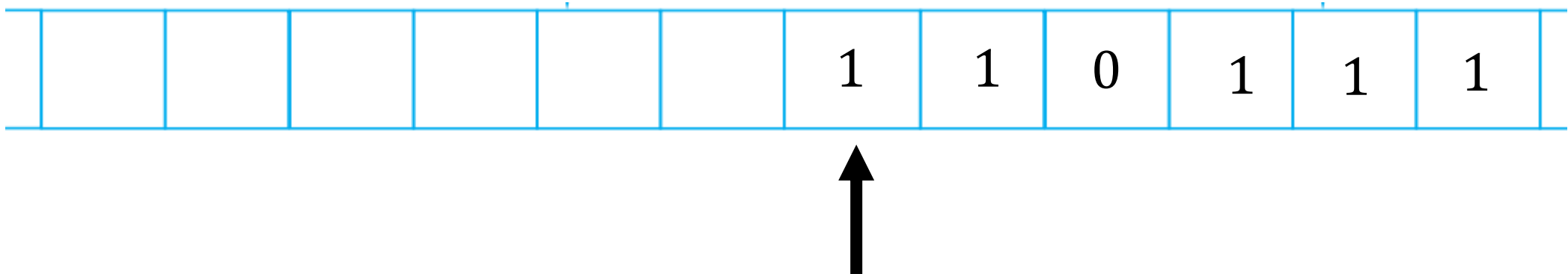
Example 9.14

- Given two positive integers x and y , design a Turing machine that computes $x \times y$.



Example 9.14

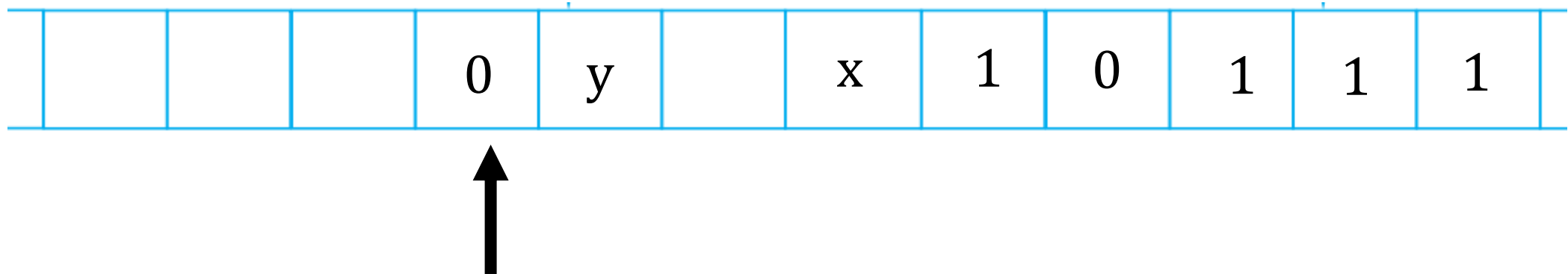
- Given two positive integers x and y , design a Turing machine that computes $x \times y$.



Turing Machine

Example 9.14

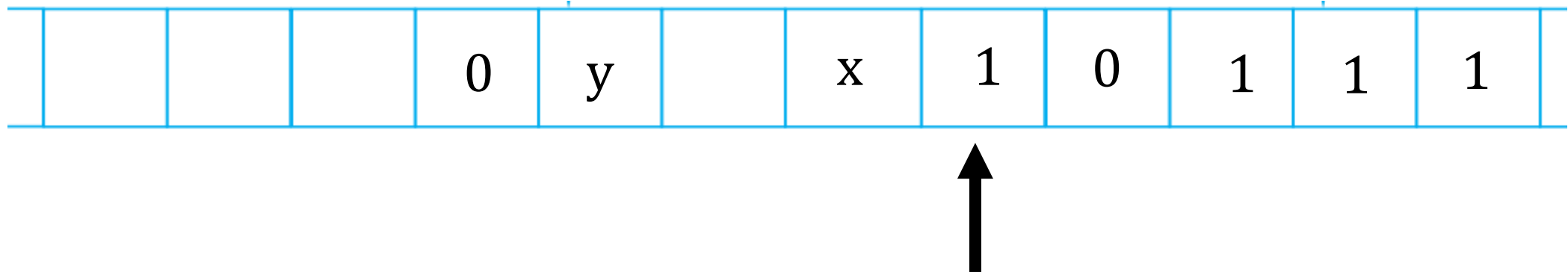
- Given two positive integers x and y , design a Turing machine that computes $x \times y$.



Turing Machine

Example 9.14

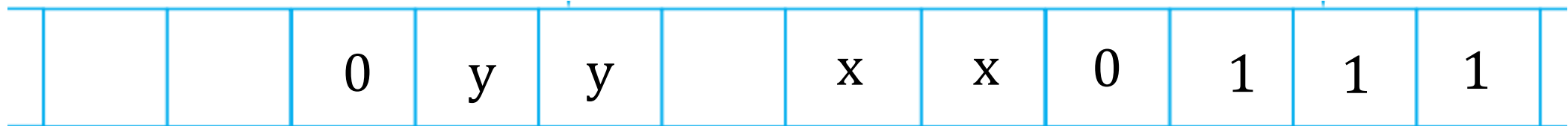
- Given two positive integers x and y , design a Turing machine that computes $x \times y$.



Turing Machine

Example 9.14

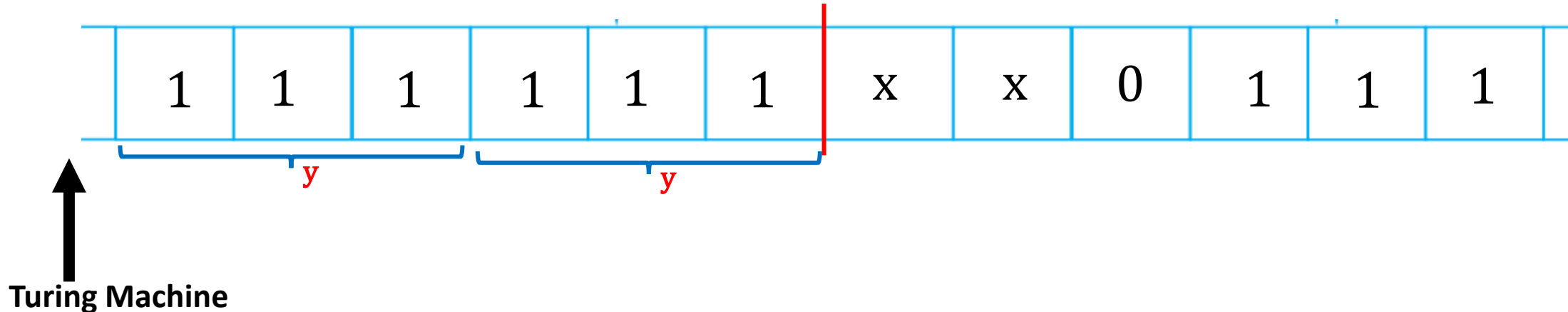
- Given two positive integers x and y , design a Turing machine that computes $x \times y$.



Turing Machine

Example 9.14

- Given two positive integers x and y , design a Turing machine that computes $x \times y$.



TURING'S THESIS

- proposed in 1936
- By Alonzo Church and Alan Turing
- "A Turing machine can do everything a real computer can do"
- "The Turing thesis remains influential, but it cannot be asserted with certainty that this thesis is correct".



What Computers Can Do?



Generate Heat

Turing Thesis

Stop a Door

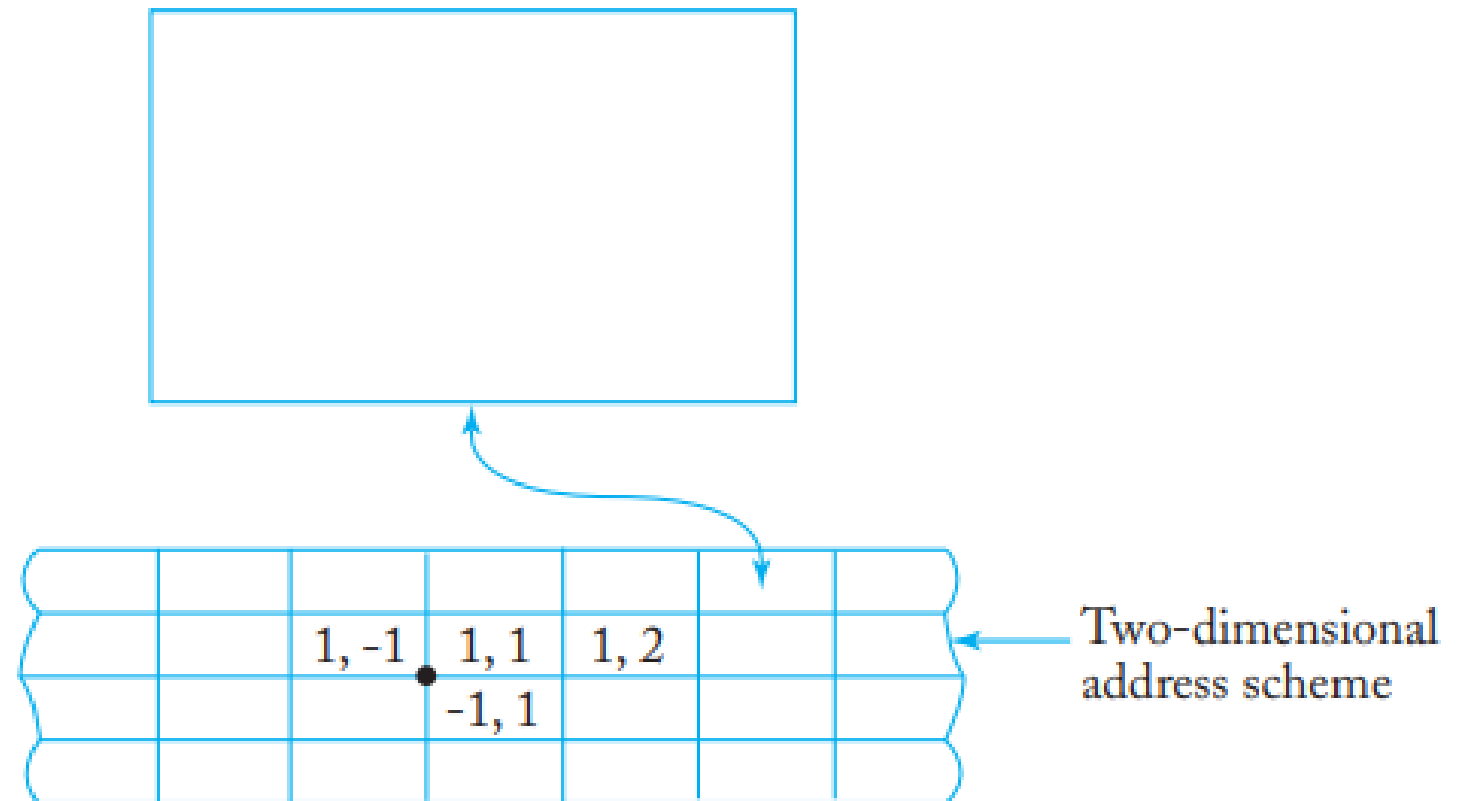


Provide an adequate
habitat for fish

2D Turing Machine

formal definition:

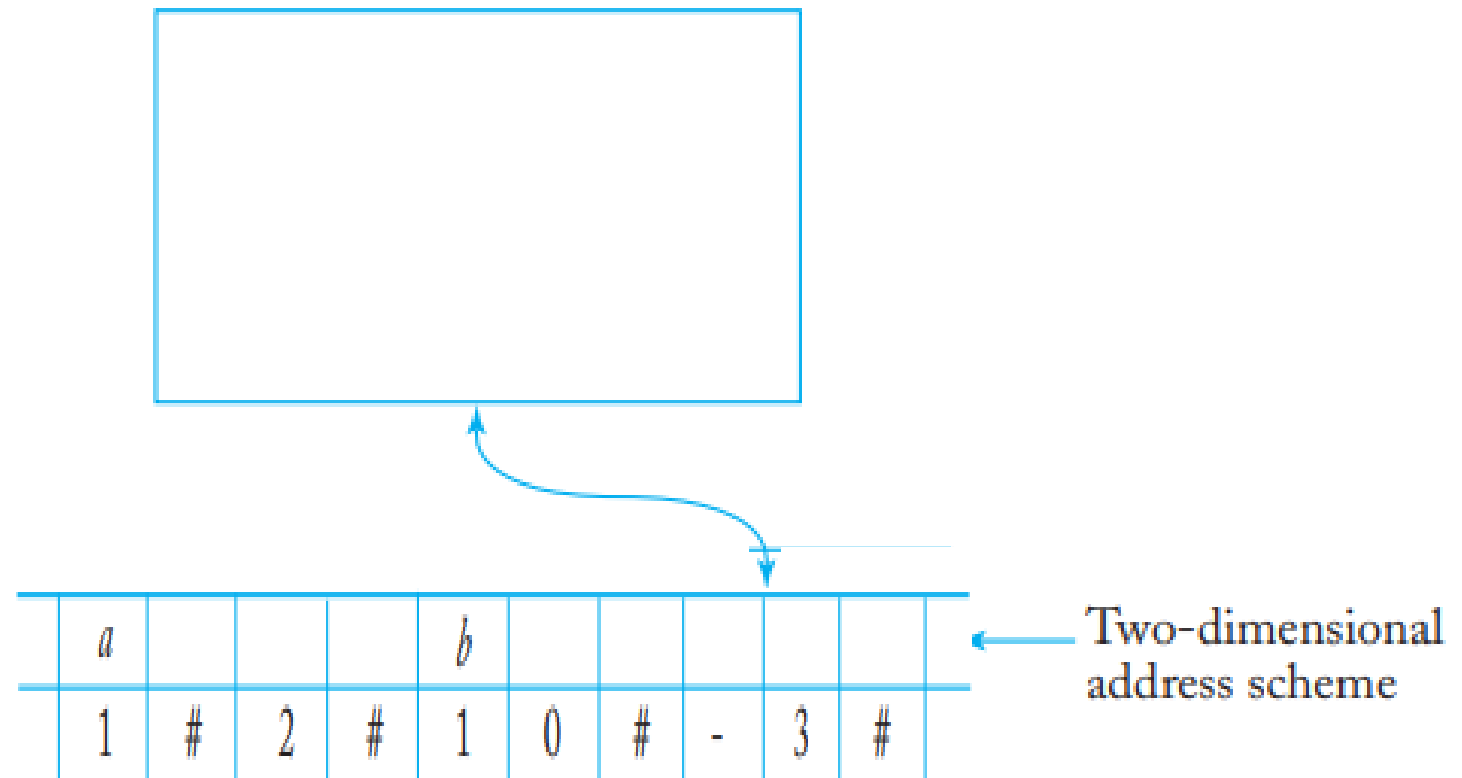
$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, U, D\}$$



2D Turing Machine

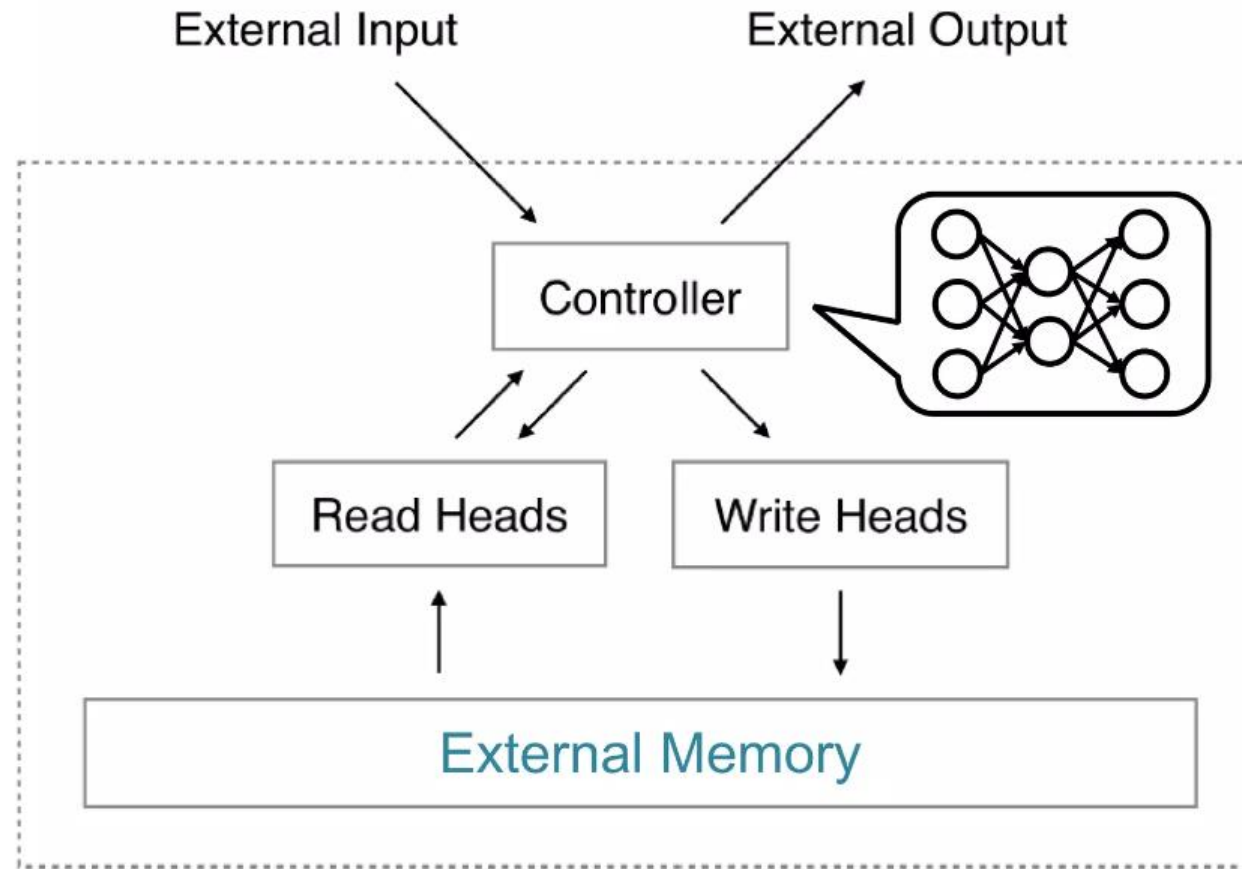
formal definition:

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, U, D\}$$



Neural Turing Machine

Neural Turing Machine



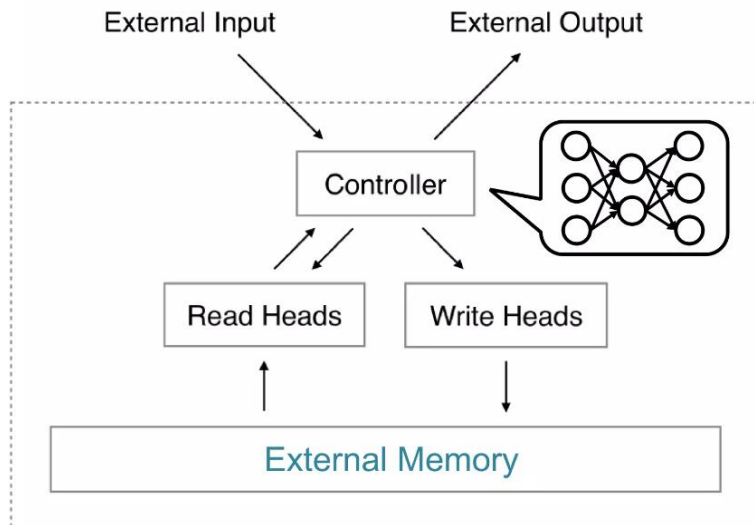
Neural Turing Machine

1. Combining Neural Networks with Sequential and Memory-based Processing:

- Strengths of Neural Networks:
- Limitations of Traditional Neural Networks:

2. External memory

- Motivation:
- Implementation:

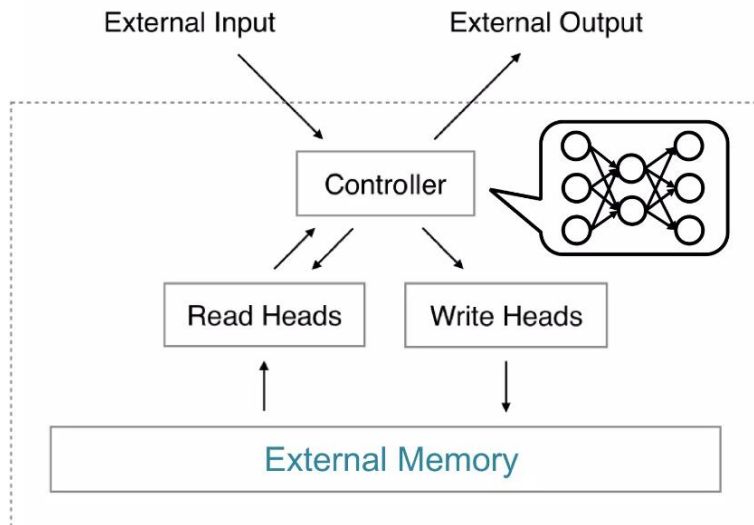


Neural Turing Machine

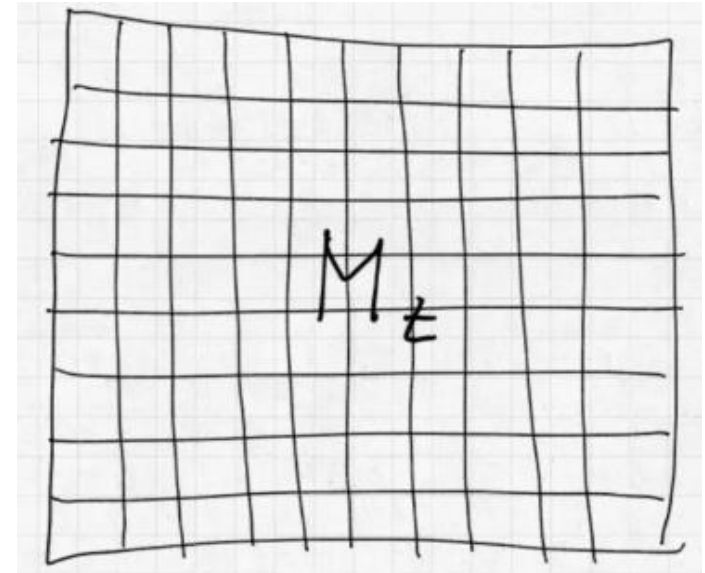
Neural Turing Machine

3. Addressing Mechanism:

- Content Addressing:
- Location Addressing:



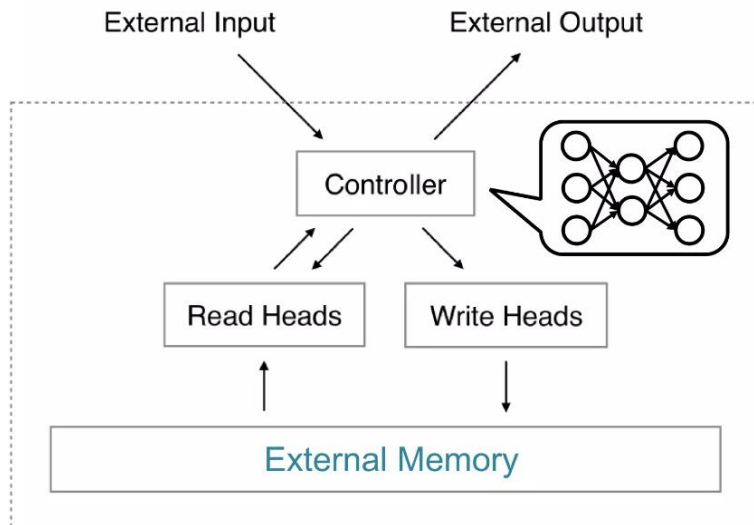
Neural Turing Machine



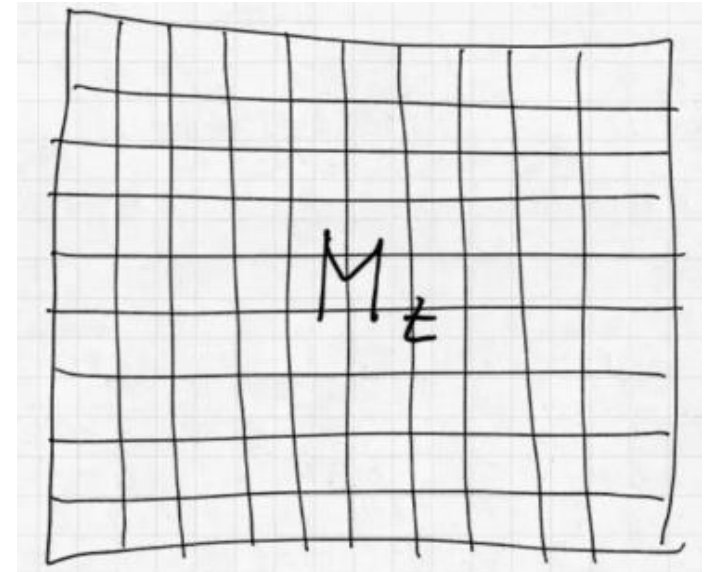
Neural Turing Machine

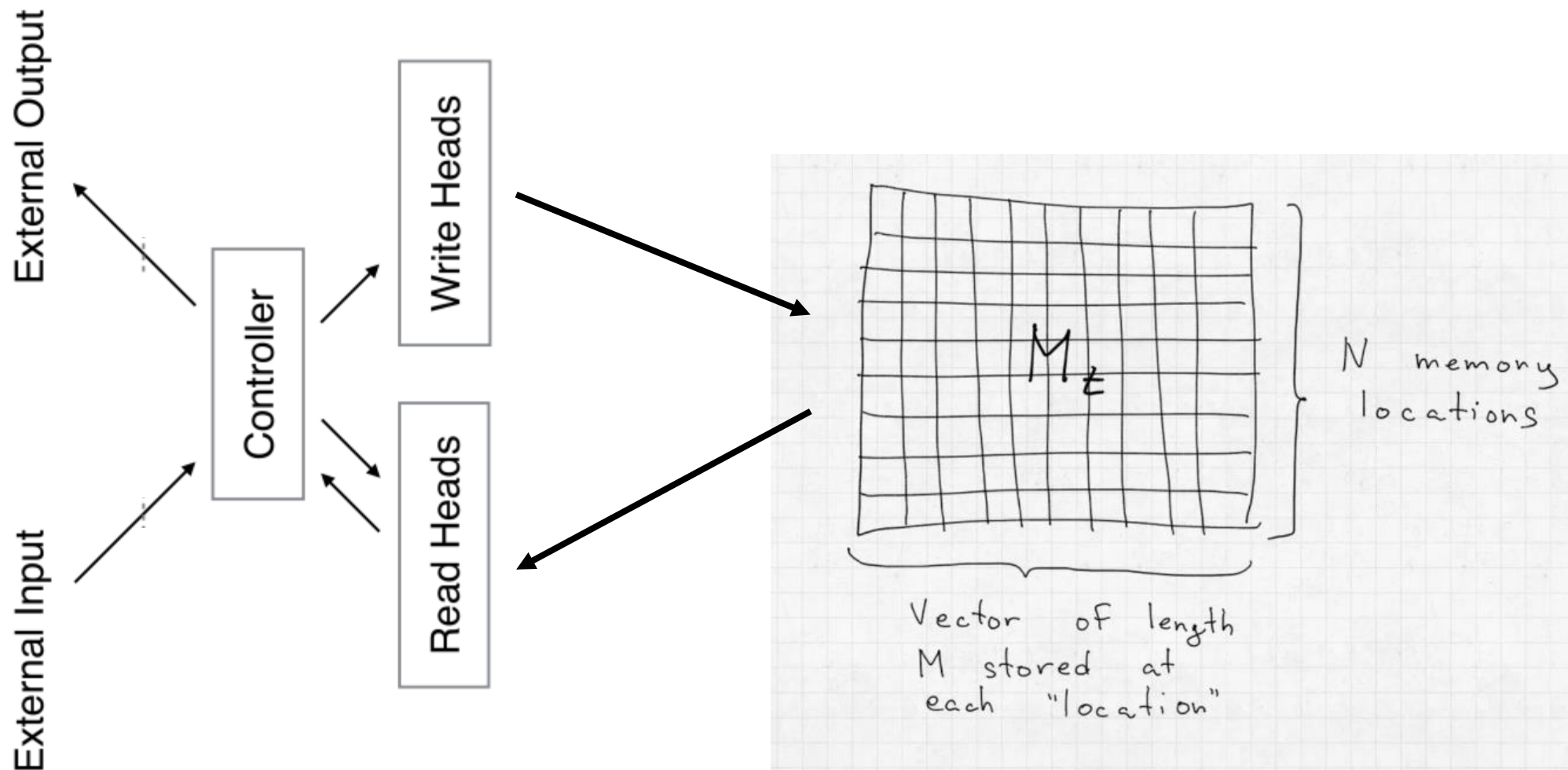
4. Training with Backpropagation Through Time (BPTT)

- End-to-End Training:



Neural Turing Machine





Neural Turing Machine

Write Head

For Erase

$$\tilde{\mathbf{M}}_t(i) \longleftarrow \mathbf{M}_{t-1}(i) [\mathbf{1} - w_t(i)\mathbf{e}_t],$$

For add

$$\mathbf{M}_t(i) \longleftarrow \tilde{\mathbf{M}}_t(i) + w_t(i) \mathbf{a}_t.$$

Read Head

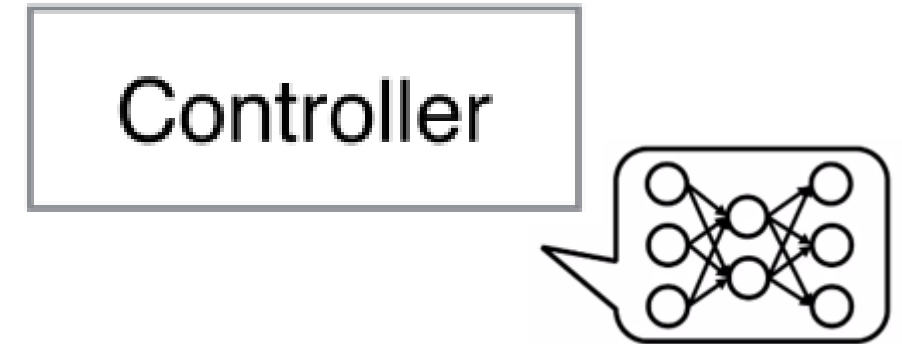
For each N Vector

$$\sum_i w_t(i) = 1, \quad 0 \leq w_t(i) \leq 1, \quad \forall i.$$

$$\mathbf{r}_t \longleftarrow \sum_i w_t(i) \mathbf{M}_t(i),$$

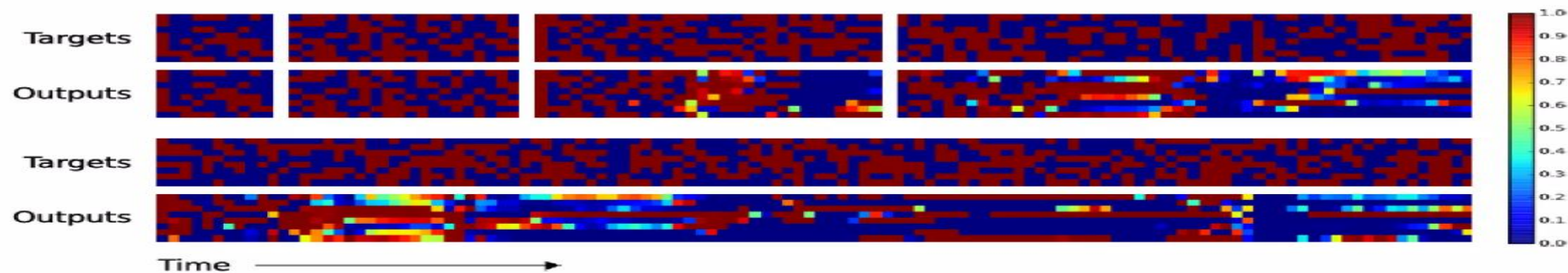
Controller Rule

1. **Neural Network Architecture:**
2. **Input Processing & Output Generation :**
3. **Learning & Adaptability :**
4. **Task-Dependent Behavior:**
5. **Addressing Mechanisms:**

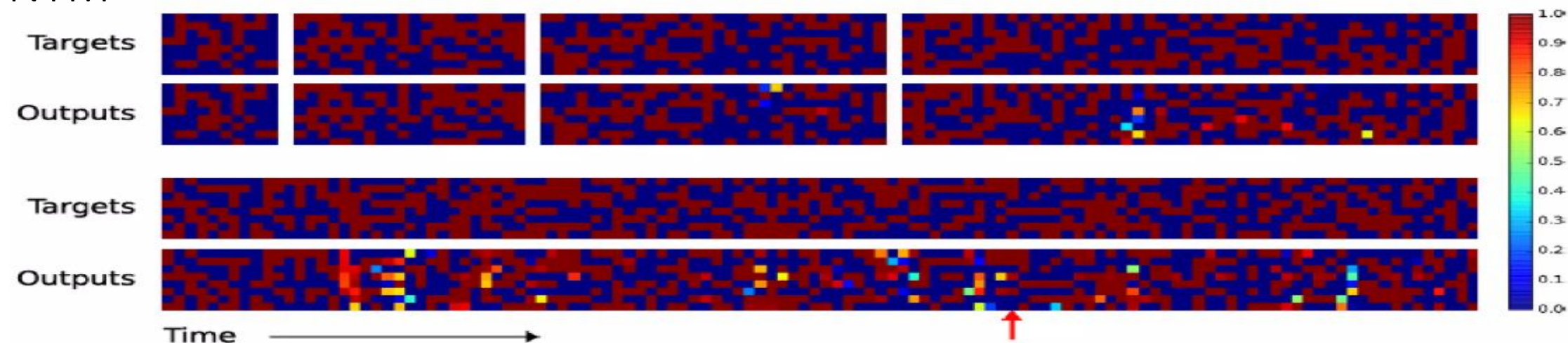


Copy

LSTM



NTM

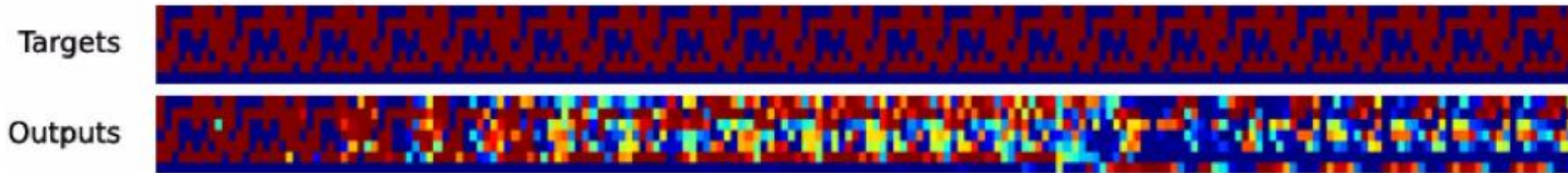


Neural Turing Machine

Repeat Copy

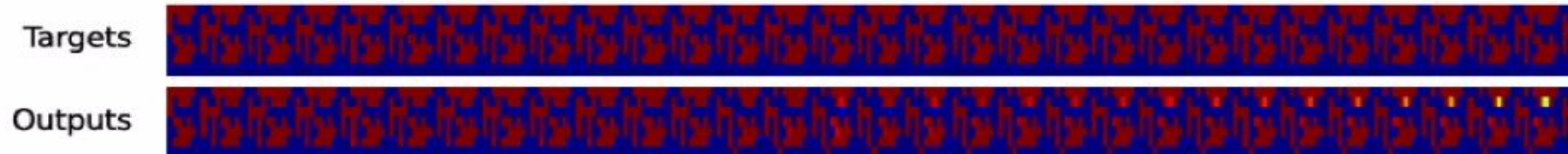
LSTM

Length 10, Repeat 20

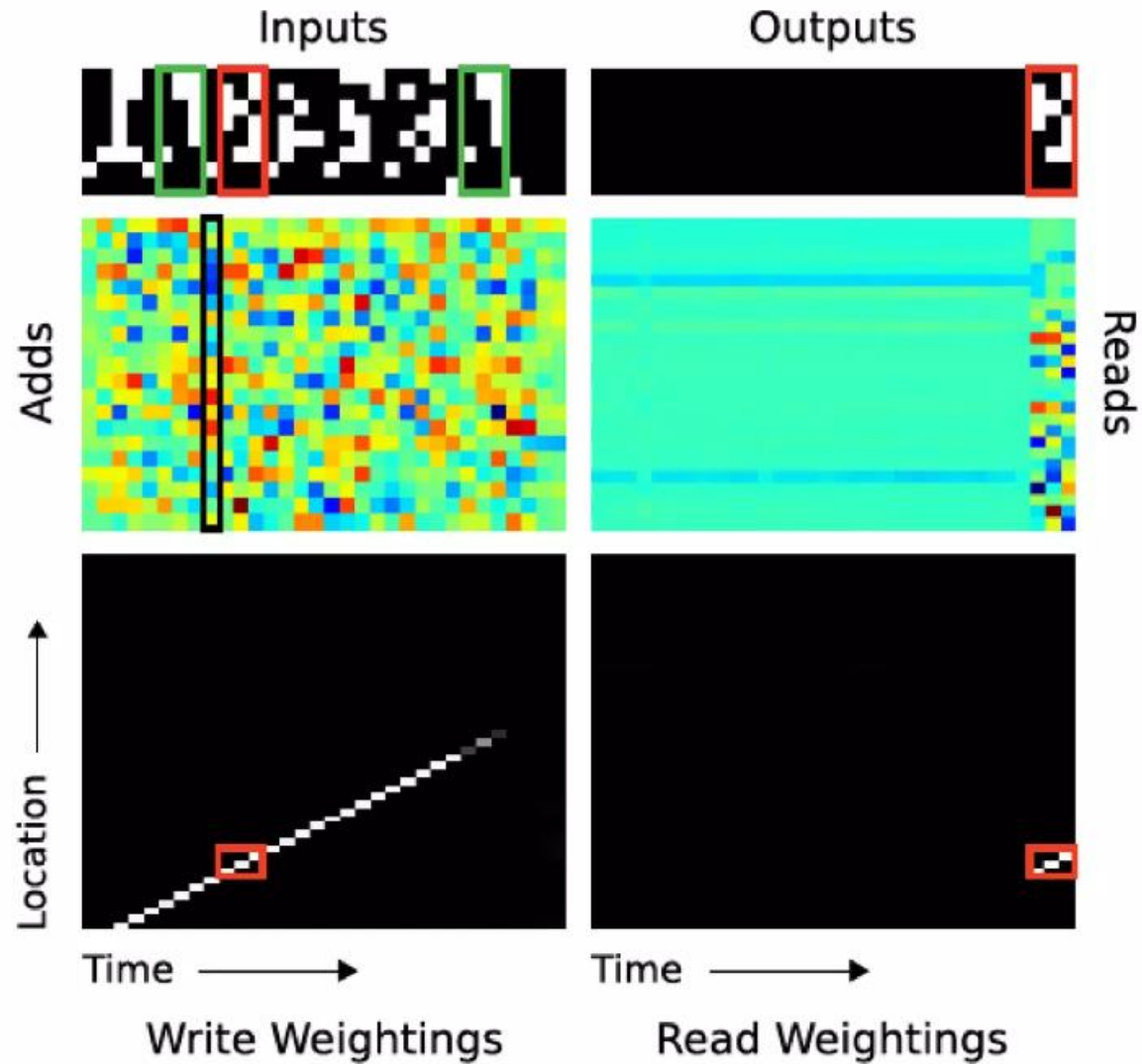


NTM

Length 10, Repeat 20



Associative Recall



Good Result!

Figure: A given QA task consists of a set of statements, followed by a question whose answer is typically a single word. [Sukhbaatar2015]

Sam walks into the kitchen.
Sam picks up an apple.
Sam walks into the bedroom.
Sam drops the apple.
Q: Where is the apple?
A. Bedroom

Brian is a lion.
Julius is a lion.
Julius is white.
Bernhard is green.
Q: What color is Brian?
A. White

Mary journeyed to the den.
Mary went back to the kitchen.
John journeyed to the bedroom.
Mary discarded the milk.
Q: Where was the milk before the den?
A. Hallway



"It's just that I've been waiting on this day for my entire life. I thought it'd feel...different."

Soul ,2020

bibliography

1. Linz , peter . an Introduction to Formal Languages And Automata . Sixth Edition . University of California at Davis.
2. sipser , Michael . Introduction to the Theory of Computation. Third Edition . Cengage Learning.
3. Kandar , Shyamalendu . Introduction to Automata Theory, Formal Languages and Computation , Pearson.
4. Sunitha , K.V.N . Formal Languages and Automata Theory . Pearson.
5. Graves , Alex . Wayne , Greg . Danihelka , Ivo . 2014 .Neural Turing Machines , Google DeepMind, London, UK.
6. Evans , David . Church-Turing Thesis Slide . Theory of Computation . University of Virginia.

Finish